GENIE
Generate, Engage, Navigate Intelligent Environments



Submitted By

Arman Adil Mangat (F2022376015),

Muhammad Suhaib Mamoon (F2022376033),

M. Farhal Raza (F2022376002),

Muhammad Huzaifa Furqan (F2022376030)

BS AI

Project Supervisor Mahmood Hussain
Department of Artificial Intelligence
School of Systems and Technology University of Management and Technology
(2022-2026)

Dedication

To Allah Almighty by whose will and blessings this work was accomplished.

To our parents who made countless sacrifices in order for us to complete this work. Late nights and early mornings they sacrificed to see us pursue our studies and graduate. For times when we missed their gathering they forgave us and did not ask us to prove what we have done was worth it. They believed in us before any of the work had been done.

To our teachers at the University of Management and Technology, Lahore Pakistan. It is them that we owe our thanks to for our chosen field of study. They taught us the difference between a measured failure and an unmeasured success and that the truth of limitations is recorded more honestly than that of convenient results. Everything we recorded as limitations fall within this idea.

To the elderly and disabled and those that care for them, to those for whom reaching a light switch, phoning for assistance, or being generally heard is not a small task.

The system was designed with you in mind, in hopes that through a living listening presence, a home may keep watch over you, assist, notify, and never ask for your life to be shared in return.

Final Approval

GENIE Generate, Engage, Navigate Intelligent Environments

Chairman of Department
Dr. Ashfaq Ahmad,
Department of Artificial Intelligence,
University of Management and Technology, Lahore.

23/06/2026

Director Final Year Projects
Mr. Basit Sattar,
Department of Artificial Intelligence,
University of Management and Technology, Lahore.

Basit 23/06/2026

Supervisor
Mr. Mahmood Hussain,
Department of Artificial Intelligence,
University of Management and Technology, Lahore.

Mahmood Hussain
Lecturer
Department of Artificial Intelligence (SST)
University of Management and Technology
23 June, 2026

Co-Supervisor
Dr. Ashfaq Ahmad,
Department of Artificial Intelligence,
University of Management and Technology, Lahore.

23/6/2026

Documentation Evaluator

Ms. Zaryab Qayyum

23/06/26

Ms. Arshi Iftikhar

Arshi Iftikhar
23/6/26

iii

Acknowledgment

All praise is due to Allah Almighty, who granted us the health, the patience and the opportunity to see this work through to completion.

We record our sincere gratitude to our supervisor, Mahmood Hussain, for his invaluable mentorship, continuous guidance across both phases of this project. His expertise in Artificial Intelligence and his dedication with the project was very important in shaping and completion of this product .

We are grateful to the Department of Artificial Intelligence and the School of Artificial Intelligence, University of Management and Technology, for the lab access, equipment and workspace that a hardware-heavy project of this kind requires.

We also thank the fellow students who volunteered their voices for the wake-word training corpus. A keyword-spotting model is only as good as the range of speakers it has heard, and their time is directly responsible for the detection rates reported in this document.

Finally, we thank our families, whose support through a long and frequently frustrating build was unconditional.

Supervised by: Mr. Mahmood Hussain
Starting Date: 01-October-2025
Completion Date: 20-September-2026

Description

Project Title: GENIE Generate, Engage, Navigate Intelligent Environments

Objective: To build a fully offline, privacy-preserving voice assistant that also performs continuous household safety monitoring, distributed across a network of ESP32-S3 microcontroller nodes, with all speech recognition, intent handling and speech synthesis executed locally on a host machine.

Undertaken by: Arman Adil Mangat (F2022376015), Muhammad Suhaib Mamoon (F2022376033), M. Farhal Raza(F2022376002), Muhammad Huzaifa Furqan (F2022376030)

Supervised by: Mahmood Hussain
Starting Date: 01-October-2025
Completion Date: 20-September-2026

Tools Used: Python 3.11; Faster-Whisper; openWakeWord/ONNX Runtime; Piper TTS; Ollama (Mistral-7B); AIML; Fast API and Uvicorn; scikit-learn; NumPy; webrtcvad; Micro Python; Arduino-ESP32 (C++); Thonny; Esp tool and MP remote; Platform IO; Git and GitHub; Jira; draw.io (C4 and UML diagrams).

Operating System: Windows 11 on the host machine; Micro Python and Arduino-ESP32 firmware on the ESP32-S3 nodes.

Documentation: Microsoft Word 2021

University of Management and Technology, Lahore

Similarity Report

Turnitin Originality Report
GENIE Generate, Engage, Navigate Intelligent Environments by Arman Adil Mangat, Muhammad Suhaib Mamoon, M. Farhal Raza and Muhammad Huzaifa Furqan

From Quick Submit (Quick Submit)

- Processed on 22-Sep-2026 15:05 PKT
- ID: 3038984762
- Word Count: 24141

Similarity Index

3%

Similarity by Source

Internet Sources:

2%

Publications:

0%

Student Papers:

2%

Sources:

1. 1% match (student papers from 24-Jan-2026)
   Submitted to Higher Education Commission Pakistan on 2026-01-24
2. < 1% match (student papers from 26-Jan-2026)
   Submitted to Higher Education Commission Pakistan on 2026-01-26

Checked by

for Verified by CLO

Note:

- Sometimes the overall similarity index may be a smaller than the repository percentages combined. This would be due to overlapping text within the repositories.
- It is a system generated report.

Declaration Form

I have carefully examined the documentation of the Final Year Project titled "GENIE Generate, Engage, Navigate Intelligent Environments"; and I endorse that this documentation complies with the standards of an undergraduate level Final Year Project report.

The document has been checked for plagiarism through Turnitin software available in UMT Library. The similarities of the document are within acceptable range.

Moreover, the accompanying CDs contain PDF of the documentation, as well as the source code and binaries with user manual and installation guide.

FYP Supervisor Name: Mahmood Hussain

Mahmood Hussain
Lecturer
Department of Artificial Intelligence (SST)
University of Management and Technology

Signature: _____ Date: 23 June, 2026

vii

Abstract

Commercial voice assistants funnel household audio through cloud servers, and this design incurs costs on the user: every interaction has network latency, basic commands stop working when the internet does, and privacy leaves the home. Home safety monitoring for smoke, gas, fire and falls is usually sold separately, as a subscription, rather than as a feature on the assistant its household owns.

GENIE (Generate, Engage, Navigate Intelligent Environments) handles both problems in a single system: an offline voice assistant and home safety monitor built on a distributed network of ESP32-S3 microcontroller nodes. Its entire AI pipeline is local. A custom-trained "Hey Genie" wake-word model (openWakeWord, exported to ONNX) gates audio capture. Faster-Whisper transcribes two independent microphone channels and keeps the more confident one. An AIML pattern kernel resolves device commands deterministically and passes only unmatched open-domain queries to a locally hosted Mistral-7B model via Ollama, and Piper synthesizes the reply. A dedicated sensor node combines two MQ-2 gas sensors, a flame sensor, a DHT11 climate sensor, two MPU-6050 inertial units and two SW-420 vibration switches. It runs a 232-parameter neural classifier on the device to recognize how the unit is being physically handled, and keeps a rule-based state machine as a baseline. None of audio, transcript or telemetry is ever sent to an outside service.

The system was evaluated by measurement, not by demonstration. Wake-word detection was tested at four distances, and false-accept rates were reported with Poisson confidence intervals. On-device inference matched scikit-learn across all 1,532 feature vectors, with a maximum confidence difference of 1.98×10^{-5}. The handling classifier reached 72.6% accuracy under a held-out-session protocol, chosen because a random split over overlapping windows would have measured memorization instead of generalization.

The paper reports two results as negative. Delay-and-sum beamforming achieved no measurable gain in detection range or false-accept rate, so it is recorded as unproven. Autonomous emergency calling worked but was disabled after the cellular module's power supply failed to sustain a 2 A GSM transmit burst, so the emergency path ships as SMS only. Capabilities not yet validated end to end are flagged as such throughout.

REVISION CHART

Version	Primary Author(s)	Description of Version	Date Completed
Draft	Muhammad Huzaifa Furqan	Initial FYP-1 documentation: problem statement, objectives, scope, domain analysis and initial product backlog.	15-12-2025
Preliminary	Muhammad Huzaifa Furqan	Second draft incorporating supervisor feedback on scope and on the requirements baseline.	20-02-2026
Final (FYP-1)	Muhammad Huzaifa Furqan	First complete FYP-1 draft submitted for evaluation.	15-05-2026
Revision 1	Muhammad Huzaifa Furqan / Muhammad Suhaib Mamoon	FYP-2 update: system architecture (C4), implementation details, algorithms and traceability matrices added.	20-07-2026
Revision 2	Muhammad Suhaib Mamoon / Arman Adil Mangat	FYP-2 update: results and evaluation added; claims reconciled against verified system state; capabilities that were disabled or dropped documented as such.	05-09-2026

Table of Contents

Dedication ii
Final Approval iii
Acknowledgment iv
Description v
Plagiarism Report vi
Declaration Form vii
Abstract viii
REVISION CHART ix
Definitions and Acronyms xiv
Table 1: List of acronyms and definitions xiv
List of Figures xvii
List of Tables xviii
1. INTRODUCTION 1
1.1 Problem Statement 1
1.2 Objectives 1
1.3 Scope of the Project 2
1.4 Significance of the Project 4
1.5 Artificial Intelligence Features 5
1.6 Project Deliverables & Hardware Prototypes 6
2. DOMAIN ANALYSIS 9
2.1 Customer 9
2.2 Stakeholders 9
Table 2: List of Stakeholders 9
2.3 Affected Groups with social or economic impact 10
2.4 Dependencies/ External Systems 11
2.5 Related Projects with feature comparisons 13
2.5.1 Related Projects 13
2.5.2 Feature Comparison 14
Table 3: Feature comparison of our product with competitors 14
2.6 Context Diagram 15
2.7 Data Flow Diagram Level 0 15
3. REQUIREMENTS ANALYSIS 16
3.1 List of Actors 16

3.2 Use Cases 17
Use Case Summary 17
Table 4: GENIE Use Case Summary 18
UC-01 Control a Household Appliance by Voice 18
UC-03 Escalate a Confirmed Emergency Without Being Asked 19
UC-04 Recognize and React to Being Handled (On-Device Edge AI) 20
UC-07 Command GENIE Remotely by SMS 22
3.3 Product Backlog 23
Figure 3: Epic vs User Story in Product Backlog [1] 23
4. PROJECT PLANNING AND EXECUTION USING SPRINTS 38
Table 5: Division of Work by Team Member 38
4.1 Jira 38
4.2 Slack 39
4.3 GitHub Repository 39
4.4 Sprint 1 39
4.4.1 Sprint 1 Planning Meeting Minutes 39
Table 6: Selected Features of Product Backlog 39
Table 7: Team Capacity Sprint 1 43
Figure 4: Class Diagram of Sprint 1 Features 44
Figure 5: Sprint 1 Sequence Diagram (Wake Word to Relay Actuation) 45
Table 10: Sprint 1 Extended Test Case 46
Table 11: Status of Sprint 1 Backlog 48
5. SYSTEM ARCHITECTURE 54
5.1 System context Diagram 54
5.1 System Container Diagram 55
5.2 Component Diagram 56
5.3 Code Diagrams 57
5.4 ERD 58
5.5 Data Dictionary 59
Table 12: Node 1 / Node 2 Telemetry Frame Fields 59
Table 13: api.py Sensor State Channels (as exposed by /sensor, /sensors) 60
Table 14: On-Device Feature Vector (har_features.py, FEATURE_VERSION 2) 60
Table 15: On-device feature vector (16 produced, 14 read by the model) 62
Table 16: Training Dataset Schema (har_data.csv) 62

6. IMPLEMENTATION DETAILS 63
6.1 Development Setup 63
6.3 Algorithms 64
6.3.1 Dual-Microphone Confidence Selection 64
6.3.2 Fall Detection Finite State Machine (Fall FSM) 65
6.3.3 Sensor Debounce, Edge Detection & Cooldown 65
6.3.4 Gas Baseline Ratio 65
6.3.5 Deterministic-First Intent Routing 65
6.3.6 Wake-Word Run-Length Gating 65
6.3.7 Voice Activity Gating with an RMS Floor 66
6.3.9 Windowed Feature Extraction for On-Device Recognition 66
6.4 Constraints 69
6.4.1 Assumptions 69
6.4.2 System constraints 69
6.4.3 Restrictions 69
6.4.4 Limitations 69
7. PROJECT MONITORING, CONTROL AND TRACEABILITY 71
7.1 Traceability Matrix 71
7.1.1 Legend Prototype / Component IDs (PID) 71
7.1.2 Legend Test Artifact IDs (TID) 71
7.1.3 Requirements vs Prototype (PB-ID vs PID) 71
Table 17: Requirements traced to implementing components (PB-ID vs PID) 75
Table 18: Requirements traced to verification (PB-ID vs TID) 77
8. RESULTS/OUTPUT/STATISTICS 78
8.1 Completion how much of the backlog is built 78
Table 21: Partial items and their blockers 79
8.2 Accuracy how well the AI components perform 79
8.2.1 On-Device Handling Classifier 79
Table 22: On-device dataset composition 80
Table 23: Handling classifier accuracy 81
Table 24: Held-out accuracy by diagnosis 81
Table 25: Confusion matrix, held-out session 81
Table 26: Confidence gate trade-off, held-out session 82
Table 27: On-device model cost 83

Table 28: Speech front end and intent layer 83
8.3 Correctness does it do what the requirements say 83
Table 29: Automated test results 84
Table 30: Structural correctness invariants 85
Table 31: Safety answer truth table 85
Table 32: Measured latency by path 85
9. CONCLUSION 86
10. FUTURE WORK 87
11. BIBLIOGRAPHY 89
11.1 Books 89
11.2 Journals 89
11.3 Articles 89
12. APPENDIX 92
12.1 Glossary of terms 92
12.2 Pre-Requisites 93

Definitions and Acronyms

Table 1: List of acronyms and definitions

Acronym	Definition
ADC	Analog-to-Digital Converter
AI	Artificial Intelligence
AIML	Artificial Intelligence Markup Language
AMR-NB	Adaptive Multi-Rate Narrowband — the speech codec used on 2G voice channels
API	Application Programming Interface
ASR / STT	Automatic Speech Recognition / Speech-to-Text
AT command	The modem command language used to control the cellular module
C4 model	A four-level notation for describing software architecture: Context, Container, Component, Code
CSFB	Circuit-Switched Fallback — handover from LTE to 2G to place a voice call
CSV	Comma-Separated Values
DFD	Data Flow Diagram
DTMF	Dual-Tone Multi-Frequency (telephone keypad signaling)
ERD	Entity Relationship Diagram
ESP-NOW	Expressive connectionless peer-to-peer wireless protocol
ESP32-S3	The dual-core Xtensa microcontroller used for every hardware node in this project
FSM	Finite State Machine
FYP	Final Year Project

GENIE	Generate, Engage, Navigate Intelligent Environments — the system described in this document
GPIO	General-Purpose Input/Output pin
GSM	Global System for Mobile communications — the 2G cellular standard
HAR	Human Activity Recognition (here: handling recognition)
HPF	High-Pass Filter
I2C / I2S	Inter-Integrated Circuit / Inter-IC Sound bus
IMU	Inertial Measurement Unit (accelerometer + gyroscope)
IoT	Internet of Things
LLM	Large Language Model
LTE	Long-Term Evolution — the 4G cellular standard
MEMS	Micro-Electro-Mechanical System — the microphone technology used (INMP441)
MLP	Multi-Layer Perceptron
NFR	Non-Functional Requirement
ONNX	Open Neural Network Exchange model format
PB-ID	Product Backlog Identifier
PCM	Pulse-Code Modulation uncompressed digital audio
PID	Prototype / Component Identifier
PSRAM	Pseudo-Static Random Access Memory
PTA / DIRBS	Pakistan Telecommunication Authority / Device Identification, Registration and Blocking System
RAG	Retrieval-Augmented Generation

RMS	Root Mean Square used here as a short-window measure of audio level
SIM	Subscriber Identity Module
SMS	Short Message Service
SMS C	Short Message Service Centre the operator relay that forwards an SMS
SNR	Signal-to-Noise Ratio
TCP	Transmission Control Protocol
TDMA	Time-Division Multiple Access the 2G transmit scheme whose 217 Hz burst rate caused audio interference
THD	Total Harmonic Distortion
TID	Test Artefact Identifier
ToF	Time-of-Flight (distance sensing)
TTS	Text-to-Speech
UART	Universal Asynchronous Receiver/Transmitter the serial link to the cellular modem
UDP	User Datagram Protocol used here for zero-configuration node discovery
UMT	University of Management and Technology
URC	Unsolicited Result Code an unprompted message emitted by the cellular modem
VAD	Voice Activity Detection
WAV	Waveform Audio File Format

List of Figures

Figure 1: Context Diagram – GENIE System..... 16
Figure 2: Data Flow Diagram (Level 0) – GENIE System..... 16
Figure 3: Epic vs User Story in Product Backlog [1]..... 24
Figure 4: Class Diagram of Sprint 1 Features..... 43
Figure 5: Sprint 1 Sequence Diagram — Wake Word to Relay Actuation..... 43
Figure 6: System Context Diagram (C4) — GENIE..... 51
Figure 7: System Container Diagram (C4) — GENIE..... 52
Figure 8: Component Diagram (C4) — GENIE Laptop Host..... 53
Figure 9: Code Diagram (Class-Level) — GENIE Sensor Pipeline..... 54
Figure 10: GENIE Runtime Data Model (no relational database is used) 55
Figure 11: GENIE development bench all nodes powered..... 62
Figure 12: Microphone node dual INMP441 wiring..... 62
Figure 13: Speaker node amplifier and speaker wiring..... 62
Figure 14: Relay node appliance wiring..... 62
Figure 15: Sensor node eight-sensor wiring. 62
Figure 16: Modem node cellular module and power feed..... 62
Figure 17: EP3-01 voice command switching a mains appliance..... 83
Figure 18: EP2-02 wake-word detection and capture in the host console..... 83
Figure 19: EP2-01 / EP2-03 transcription, channel selection and intent routing..... 83
Figure 20: EP3-02 sensor hazard alert reaching the host..... 83
Figure 21: EP3-03 emergency SMS received on a handset..... 83
Figure 22: Wake-word detection rate measurement setup..... 83
Figure 23: On-device activity classifier live output and confidence gate..... 83

List of Tables

Table 1: List of acronyms and definitions 13
Table 2: List of Stakeholders 9
Table 3: Feature comparison of our product with competitors 13
Table 4: GENIE Use Case Summary 19
Table 5: Division of Work by Team Member 38
Table 6: Selected Features of Product Backlog. 39
Table 7: Team Capacity Sprint 1. 41
Table 8: Sprint 1 Backlog 42
Table 9: Sprint 1 Decision Table — Recording Window Activation. 44
Table 10: Sprint 1 Extended Test Case 44
Table 11: Status of Sprint 1 Backlog. 45
Table 12: Node 1 / Node 2 Telemetry Frame Fields 55
Table 13: api.py Sensor State Channels (as exposed by /sensor, /sensors) 56
Table 14: On-Device Feature Vector (har_features.py, feature version 2) 57
Table 15: On-device feature vector (16 produced, 14 read by the model) 58
Table 16: Training Dataset Schema (har_data.csv) 58
Table 17: Requirements traced to implementing components (PB-ID vs PID) 72
Table 18: Requirements traced to verification evidence (PB-ID vs TID) Coverage summary 74
Table 19: Backlog completion by state 75
Table 20: Derived completion figures 75
Table 21: Partial items and their blockers 76
Table 22: On-device dataset composition. 77
Table 23: Handling classifier accuracy 77
Table 24: Held-out accuracy by diagnosis 78
Table 25: Confusion matrix, held-out session 78
Table 26: Confidence gate trade-off, held-out session. 79
Table 27: On-device model cost 79
Table 28: Speech front end and intent layer 80
Table 29: Automated test results 81
Table 30: Structural correctness invariants 81
Table 31: Safety answer truth table 82
Table 32: Measured latency by path. 82

1. INTRODUCTION

The popularity of voice-controlled home automation has risen exponentially in the past decade, driven by the emergence of commercial assistants such as Amazon Alexa and Google Home, which enable a person to operate lights, appliances and information services by means of spoken commands. Meanwhile, the broader IoT ecosystem has also developed, allowing individuals to construct their own sensor networks and home automation devices using low-cost microcontrollers like the ESP32-series and eliminating the need for proprietary commercial systems.

However, nearly all mainstream voice assistants relay audio and command processing to clouds, which introduces three ongoing problems for the home user: perpetual transfer of private household audio to third-party entities, dependence on an active internet connection even for simple commands, and increased response latency. In addition, home safety monitoring (fire, smoke, gas, and fall detection) is usually a separate subscription service, rather than being incorporated into the same assistant that a household uses.

GENIE (Generate, Engage, Navigate Intelligent Environments) is a final-year project which combines two fields together: a personal voice assistant and a home safety-monitoring system, which is based on a distributed network of ESP32-S3 microcontrollers, with its entire AI pipeline (speech recognition, intent handling, speech synthesis) running locally on the host machine, independent of any cloud service.

1.1 Problem Statement

Conventional smart home voice assistants are reliant on cloud computing, resulting in latency, privacy concerns, and a dependence on an Internet connection. In addition, standard assistants do not provide native rule-based control for instant hardware response or dedicated embedded safety monitoring (flame, smoke, and fall detection) that is tightly integrated with interactive video graphics. Microcontroller designs that manage multiple high-bandwidth digital audio streams, display drivers, and environmental sensors suffer from severe pin conflicts, power distribution problems, and resource allocation issues.

1.2 Objectives

- **Distributed Embedded Processing:** Distribute physical I/O operations of audio capture/playback, relay switching, and sensing over dedicated ESP32-S3 boards to avoid hardware resource conflicts
- **Local AI Processing Pipeline:** Run wake-word-gated Speech-to-Text (Faster Whisper), Intent Routing (AIML + Mistral-7B via Ollama), and Textto-Speech (Piper) processing fully offline on a local host machine
- **Wake-Word Gated Activation:** Utilize a custom-trained "Hey Genie" keyword-spotting model (openWakeWord, exported to ONNX) to constantly listen (at lower cost) incoming audio, and trigger full speech capture when the wake phrase is recognized

- **Dual-Microphone Audio Streaming:** Use simultaneous MONO I2S digital audio streams to improve transcription confidence in different acoustic conditions
- **Integrated Safety & Smart Control:** Integrate active-LOW household relay control plus dual-redundant safety monitoring (gas, flame, fall detection, and vibration sensing) into a two-node sensor architecture that also includes a time-of-flight presence array, along with dynamic expressive visual feedback (RoboEyes ILI9341 display)
- **Zero-Configuration Networking:** Use UDP auto-discovery to allow Wi-Fi enabled microcontroller nodes (microphone, speaker, and relay boards) to connect to the host Python server, without having to hardcode IP addresses

1.3 Scope of the Project:

- **Distributed Microcontroller Architecture:** Hardware is spread over five separate ESP32-S3 boards, each handling a particular task: capturing audio from two microphones, playing audio over a speaker, controlling relays for home appliances, and managing a two-node safety/presence subsystem. Microphone, speaker, and relay boards talk over a local Wi-Fi network (GENIE1), while the two safety/presence nodes exchange sensor data over ESP-NOW and report to the host over a direct USB-serial connection.
- **Local Offline AI Processing Pipeline:** A Windows laptop is running a fully offline software pipeline, including speech capture gated by a wake word via AudioServer.py, local transcription using Faster-Whisper, intent classification using an AIML kernel, open-domain fallback conversations handled by a local Mistral-7B LLM on Ollama, and speech synthesis using Piper TTS.
- **Dual-Microphone Audio Streaming & Confidence Selection:** MONO digital I2S audio capture from two INMP441 MEMS microphones while streaming to the host as a single interleaved dual-channel connection on TCP port 9000 (port 9003 is now reserved as a dedicated control channel). Transcription confidence comparison (avg_logprob) is used to dynamically select the clearer audio input.
- **Dynamic Node Discovery:** A zero-configuration UDP auto-discovery protocol, operating on port 9999, is used by the Wi-Fi-connected microcontroller nodes (microphone, speaker, and relay boards) to dynamically find and connect to the host laptop's IP address on boot.
- **Smart Home Appliance Actuation:** A web-based HTTP GET control interface (/relay? Light=<0|1>&fan=<0|1>) on an active LOW relay board is used to toggle home appliances (light and fan) per classified voice command.
- **Safety & Environmental Monitoring, Two-Node Redesign:** A dual redundant Node 1 (face/sensor board) runs two MQ-2 gas sensors, a flame sensor, a DHT11 temperature/humidity sensor, two MPU-6050

fall detection IMUs, and two SW-420 vibration sensors, while Node 2 (presence board) runs a four-point VL53L1X time-of-flight array for room presence sensing, supplanting the earlier single IR-obstacle presence sensor. Node 2 communicates with Node 1 by ESP-NOW and Node 1 communicates with the host by serial JSON telemetry channel, which is now fully incorporated into the host-side API (a TCP board listener plus bridging module translates the readings into the API's sensor state), leaving only hardware bring-up to do, since an intermittent power-supply issue is currently causing the ToF sensors to fail unpredictably at boot.

- **Expressive Visual Feedback:** Run an onboard ILI9341 LCD shield with animated "Robo Eyes" that adapt their mood expressions in real time according to sensor data and alert conditions.
- **System Safety Guardrails:** Whisper noise filtering (discard threshold set at no_speech_prob > 0.6) plus LLM system-prompt guardrails are used to stop hallucinated sensor data or invalid safety claims.
- **Emergency Communication Module:** An automated emergency alert system using the GSM-based A7670E module to call and SMS when a critical safety event like a fall, flame, or gas leak is verified over a dedicated TCP connection; the modem firmware and trigger logic have been installed, but end-to-end live testing is still due, including SIM network registration (PTA/DIRBS) and hardware bring-up.
- **Web Dashboard (Planned):** A web-based sign-in dashboard for remote review of system/sensor status one of the project's remaining evaluation milestones; work has not begun.
- **Battery-Powered Operation (Planned):** Untethered, battery-milestone; work has not begun.
- **Visual Monitoring via ESP32-CAM (Planned):** Camera-based visual verification to complement the sensor-based alerts architecture is defined, implementation is pending.
- **On-Device Edge AI (Handling Recognition):** A machine-learning classifier running locally on the ESP32-S3 itself, rather than the laptop. A 14-12-4 multi-layer perceptron (232 parameters) is trained off-line by MPU-6050 windows and exported to plain Micro Python source, allowing the face board to detect taps, pushes, pick-ups, or shakes without TensorFlow Lite, NumPy, or a custom firmware build. A hand-tuned finite state machine (handling_fsm.py) is kept alongside it as the rule-based baseline that the backed operation of the hardware nodes the project's other remaining evaluation learned model must outperform.
- **Autonomous Climate Control:** A self-contained control loop which monitors the DHT11 and independently activates the fan relay when the room crosses a set temperature or humidity threshold, announcing the specific reading which triggered it. A user's spoken command takes precedence over the rule until the environmental conditions themselves change.

- **Conversational Follow-Up Window:** An eight-second period following GENIE's speech during which only the voice-activity detector may start a recording, allowing a follow-up question to be asked without another wake word. The conversation chain is limited to three exchanges before the wake word must be used again.
- **Two-Way SMS Command Relay and DTMF Acknowledgement:** Any SMS from an approved number arriving at the modem node is forwarded to the host, processed through the same intent pipeline as a spoken command, and sent back as an SMS reply. During an emergency call, the contact can press 1 to acknowledge and halt the escalation, or press 2 to forward it to the next contact.
- **Beamformed Dual-Microphone Front End:** The two microphones are combined sample by-sample into one detection channel (delay-and-sum, broadside), which the wake word and VAD listen to; each channel is also recorded separately so that Whisper can transcribe both and the transcript with higher confidence is selected.
- **Development and Verification Tooling:** A hardware-free simulator (fake_node.py, twelve board scenarios), a live terminal monitor (sensor_monitor.py), a standalone serial bridge (serial_bridge.py), and a preflash consistency checker (check_pack.py), enabling the entire pipeline to be rehearsed and regression-tested without any boards attached.

1.4 Significance of the Project

The GENIE (Generate, Engage, Navigate Intelligent Environments) project addresses several issues in current consumer smart home solutions related to privacy, latency, robustness, and multi-modal safety. By offloading voice processing and IoT coordination to a local distributed embedded framework, this project provides benefits to society, the industry, and academia:

Effects on Society and Households:

- **Privacy Safeguarding:** Entirely disposes of the constant transfer of private household audio and telemetry to third-party cloud servers, thus protecting the user data with a guarantee of privacy and shielding them from potential remote surveillance or breaches.
- **Improved Home Safety and Accessibility:** Provides with the ability to monitor dangerous situations, such as the appearance of smoke or gas in real-time without delays to react and take action. Moreover, the addition of expressive visual feedback ("Robo Eyes") enables intuitive communication of the robot's status, useful to the elderly or children
- **Independence from the Internet:** Keeps essential voice command execution and emergency safety functions operational even when the internet connection is down, thus preserving the home automation reliability.

Industrial & Applied Engineering Impact:

- **Cost-Effective Modular IoT Architecture**: Shows how affordable, off the shelf microcontroller (ESP32-S3) can be arranged into specific node topologies to circumvent hardware pin resource conflicts and reduce per-unit manufacturing costs over centralized custom PCBs
- **Zero-Configuration Deployment:** Utilizes dynamic UDP auto-discovery protocols to facilitate a streamlined user experience, circumventing the need to utilize complex static IP management or manual network updates.

Academic & Research Contribution:

- **Local Multi-Modal AI Pipeline:** Acts as a practical benchmark for coordinating multi-tiered AI inference pipelines (Faster-Whisper for STT, hybrid AIML rule engines combined with local 7B-parameter LLMs, and Piper TTS) that execute simultaneously on consumer-grade local hardware.
- **Audio Confidence Optimization:** Confirms a real-time, dual-microphone array configuration that selects between audio input streams based on transcription log-probability scores, providing a valuable model for local acoustic signal selection.

1.5 Artificial Intelligence Features

GENIE applies AI at several distinct points in its processing pipeline, rather than relying on a single cloud service:

- **Wake-Word Detection:** A custom wake-word model (openWakeWord, exported to ONNX) trained for "Hey Genie" watches the incoming audio stream with minimal CPU load, waking the full speech pipeline only on recognition of the wake phrase.
- **Offline Speech Recognition:** Faster-Whisper runs captured audio on device through a no_speech_prob confidence threshold to filter out background noise and false triggers before passing to the intent layer.
- **Hybrid Intent Routing:** transcribed commands are first tested against an AIML pattern-matching engine for deterministic, low-latency device control, and queries that fail that test are passed to a locally-hosted Mistral-7B LLM (via Ollama) for open domain handling. Sensor-related questions are specifically excluded from this fallback and are always sent to the deterministic intent layer instead the LLM's system prompt explicitly forbids it from reporting any sensor reading, so it can't fabricate an answer (like a false "no smoke detected") that the sensors never gave.
- **Rule-Based Safety Fast-Path:** critical hazard readings (flame, gas threshold breach, fall detection) bypass the full ASR/LLM pipeline and go straight to rule-based relay actions, guaranteeing that safety-critical response time isn't compromised by AI inferencing latency.

- **Natural Speech Synthesis:** Piper TTS turns AIML/LLM responses into natural-sounding offline speech, closing the voice interaction loop without requiring the cloud.
- **Planned Visual Recognition (ESP32-CAM):** a future feature intended to supplement the audio- and sensor-based interaction with basic on device visual checks and monitoring.
- **On-Device Edge AI (Handling Recognition):** a supervised classifier trained on the laptop from real accelerometer data to extract features, and then running itself on the ESP32-S3. The same feature-extraction file (har_features.py) is imported by both the training script and the board, so the numbers used to fit the model and the numbers it sees can't be different. The exported model is 232 plain numbers plus a pure Micro Python predict (); a version stamp is checked at import so a mismatched model fails loudly rather than silently reading the wrong column.
- **Learned vs. Rule-Based Comparison:** the same five handling events are also generated by a hand-tuned finite state machine (handling_fsm.py). Keeping both allows the report to state what the learned model actually gives over thresholds written in an afternoon, rather than assuming it gives something.
- **Honest Generalization Measurement:** the classifier is tested twice once on a random split of one recording session, and once on a second session recorded on a different day and never seen during training. Only the second number is cited in Chapter 8, and the difference between the two is reported.
- **Abstention Rather Than Guessing:** the board says nothing at all when the model's confidence drops below 0.70. Silence is a valid output, as the same principle dictates that an unreadable sensor report would say "unknown" rather than "clear".
- **Autonomous Environmental Control:** a single-decision rule based on temperature and humidity that controls the fan relay and explains itself out loud, with two-threshold hysteresis so a room hovering on a boundary doesn't toggle the relay every poll.

1.6 Project Deliverables & Hardware Prototypes:

- **Microphone Node Prototype:** A working ESP32-S3 hardware with two INMP441 MEMS microphones each with separate I2S channels for streaming audio.
- **Audio Speaker Node Prototype:** An audio speaker node prototype working ESP32-S3 + MAX98357A I2S Class-D amplifier + 5W 4Ω speaker for local text-to-speech playback
- **IoT Appliance Controller Node Prototype:** An IoT appliance controller node prototype working ESP32-S3 active LOW relay node which can control a number of high-voltage home appliances (tested with a light and a fan) with a built-in WS2812 status LED.

- **Safety & Sensor Node Prototype (Node 1):** A safety/sensor node prototype (node 1) a working ESP32-S3 hardware with twin MQ-2 (gas/smoke) sensors a flame sensor a DHT11 temp/humidity sensor twin MPU6050 fall detection sensors and twin SW-420 vibration sensors plus a parallel ILI9341 LCD for displaying the animated "Robo Eyes"
- **A presence sensing node prototype (node 2):** a working ESP32-S3 hardware with a four-point VL53L1X time-of-flight array for presence sensing which communicates with node 1 via ESPNOW the latter has been tested standalone and is awaiting inclusion in the main loop of node 1's firmware

Software application and local AI pipeline:

- **Local host audio and AI orchestrator:** a Python server which handles multi-port TCP audio capture (see AudioServer.py) auto-discovery of network nodes via UDP and Piper TTS audio generation
- **Speech processing pipeline + intent recognition:** local Faster-Whisper pipeline for speech-to-text diarization with log probability confidence selection based on dual-microphone input a compiled AIML rule-based intent router and a local (offline) Mistral-7B LLM (via Ollama) with safety filters
- **API and telemetry management:** Fast API backend (api.py) which is now self-sufficient in terms of sensor operations: it exposes a /relay endpoint for actuating the ESP32 relays a TCP listener for the board telemetry and a UDP discovery broadcast which serves as an entry point for the Node 1/Node 2 sensor readings which are then processed into a unified stream for the AIML pipeline.

Project Documentation & Test Artifacts:

- **Comprehensive System Documentation:** Final project report describing system architecture, hardware schematics, pinouts, network communications, and software module responsibilities
- **Test Cases & Validation Suite:** Described unit and integration tests which verify confidence thresholds for audio routing, relay switching delays, threshold-based sensor alerts, and offline AI processing accuracy
- **User & Deployment Guide:** Technical manual which details local software installation, hardware assembly, firmware flashing procedures, and network setup for access points.

Planned / In-Progress Deliverables:

- **Emergency Communication Module:** GSM-based (A7670E) modem firmware and TCP trigger logic are implemented and unit-tested; live end to-end validation is pending SIM network registration (PTA/DIRBS) and hardware bring-up.
- **Web Dashboard (Planned**): browser-based sign-in dashboard for remote status review remaining project evaluation gate, not yet started.
- **Battery Powered Operation (Planned):** untethered, battery-backed hardware operation remaining project evaluation gate, not yet started.

- **ESP32-CAM Visual Monitoring Module:** camera-based visual verification to complement sensor-based alerts; architecture defined, implementation pending.
- **Node 1 / Node 2 Hardware Bring-Up:** Node 2 is merged into Node 1's firmware loop and the host side API fully ingests the new serial/TCP telemetry channel (code complete, 86 automated assertions passing). What remains is hardware bring-up: an intermittent power-supply issue is causing the ToF array to fail non-deterministically at boot (ENODEV) and is suspected in a display panel reset storm on Node 1 as well.

Edge AI Artefacts:

- **Labelled On-Device Dataset:** har_data.csv 1,532 labelled feature windows collected from two recording sessions performed on separate days, each including six classes (idle, tapped, pushed, turned, picked up, shaken) by the user under test, with the session ID and feature-vector version stored in each row.
- **Shared Feature Extractor:** har_features.py one implementation of the 16-value feature vector, usable unchanged in both Micro Python and Python, imported by the board and the trainer so that train/test skew is avoided structurally rather than just accidentally.
- **Training and Export Pipeline:** har_train.py trains and evaluates both a decision tree and an MLP, reporting a random-split and a session-held-out accuracy in each case, and exports the tree and MLP as Micro Python source code.
- **Exported On-Device Model:** har_model.py the generated 14-12-4 MLP with frozen standardization constants, plus a predict () function in pure Micro Python, returning the predicted class and the associated SoftMax confidence.
- **Data-Collection Harness:** har_record.py a labelling tool which labels whatever gesture the user is performing when the board is streaming, and confirms that vectors are being received before requesting that the user performs any gesture.
- **Rule-Based Baseline:** handling_fsm.py the hand-crafted finite-state machine which serves as the baseline for comparison against the learned models.

2. DOMAIN ANALYSIS

2.1 Customer

GENIE caters to a broad audience, ranging from the average homeowner to the specific needs of individuals with unique requirements. While it was initially designed to address the needs of physically disabled individuals, the application has evolved to meet the demands of a broader audience of smart home users. The target audience for this product includes ordinary homeowners and tech-savvy consumers, who want to be able to manage home appliances, such as fans and air conditioners, with voice commands while maintaining data privacy. Consumers, who desire to be able to control all home appliances in their hands while having a hands-free experience, will also find this product useful. Hands-free control of home appliances is particularly beneficial for elderly people or those with disabilities or impairments. Additionally, GENIE is appropriate for families, as it provides security by constantly monitoring the home environment for dangers, such as smoke, combustible gases, indoor fires, and falls. With its hands-free control of home appliances, local conversation, and instant safety hazard alerts, GENIE is an appropriate virtual assistant for modern smart homes, which can be accessed by a diverse audience with various demands and levels of experience.

2.2 Stakeholders

The GENIE system uses a number of critical stakeholders, ranging from end-users interacting with the hardware and voice assistant to developers and maintainers of the system at large. Taking them into consideration ensures that needs such as privacy, hands-free interaction, safety alerts, and maintenance are accounted for and adequately addressed in the creation of the hardware-software interface.

Table 2: List of Stakeholders

Stakeholder	**Role in System**
Homeowners / General Users	Issue verbal commands to operate home appliances such as lights and fans. Engage in open-domain dialogues with the local AI model. Monitor and review environmental data and status reports presented on the "Robo Eyes" interface.
Elderly & Physically Impaired Users	Use Voice commands to operate home appliances hands-free and without a separate switch or app Get expressive visual feedback ("Robo Eyes") showing operation status, safety alerts and more conditions.

Household & Members Caregivers	Benefit from real-time environmental safety monitoring for household dangers (smoke, combustible gas, home fire). It Receives immediate visual alerts in the case of physical falls and safety hazard detections.
Development Team (System Creators)	Design, build and integrate the distributed ESP32-S3 microcontroller hardware nodes Program and optimize the local offline AI pipeline (Faster-Whisper, AIML intent engine, Ollama/Mistral-7B LLM, and Piper TTS) Configure local network routing, Fast API telemetry endpoints, and low-latency audio/sensor streaming protocols.
System Administrator / Technical Maintainer	Manage local network configuration (Wi-Fi access point setup and dynamic UDP discovery). Flash firmware updates to microcontroller boards and update software dependencies on the host PC. Maintain system uptime, monitor local resource utilization, and perform offline model updates.

2.3 Affected Groups with social or economic impact

General Homeowners & Tech-Conscious Users

- **Social Impact:** Guarantees total data privacy by conducting all voice processing and conversational artificial intelligence procedures locally and eliminating potential risks of third-party data surveillance and interception.
- **Economic Impact:** Offers an affordable solution for smart home automation by utilizing a modular ESP32-S3 rather than an expensive proprietary system, significantly cutting down on the economic threshold for the targeted audience.
- **Link to Objectives:** A direct result of implementing an offline local AI pipeline (Faster-Whisper, AIML, Ollama) and inexpensive distributed microcontroller architecture.

Elderly & Mobility-Impaired Individuals

- **Social Impact:** Encourages a higher level of independence while improving the quality of life by introducing a hands-free voice-controlled interface to operate the home appliances and nonverbal user feedback ("Robo Eyes") for a better user experience.
- **Economic Impact:** Lowers financial and care burdens, by automating household chores and environmental monitoring, to relieve the need for full-time physical caregivers.

Families & Household Members

- **Social Impact:** Domestic peace of mind through real-time local monitoring of major household dangers, including smoke, combustible gas, fire, and falls.
- **Economic Impact:** Avoidance of pricy property destruction and medical expenses through early environmental hazard detection and immediate localized warning.
- **Link to Objectives:** Meets the goal of integrating active-LOW relay actuation with dual redundant, multi-sensor safety monitoring and time-offlight presence detection in the Node 1/Node 2 safety subsystem.

System Installers & Technical Maintainers

- **Social Impact:** Deployment ease and reduced user frustration through the elimination of the need for users to manually configure static IP addressing and routing rules.
- **Economic Impact:** Reduction in labor and support costs associated with device installation, setup troubleshooting, and network reconfiguration.
- **Link to Objectives:** Directly supported by the UDP auto-discovery protocol on port 9999, which supports dynamic IP binding.

2.4 Dependencies/ External Systems

The GENIE system is built upon open-source local AI frameworks, core python runtimes, hardware drivers, and localized network infrastructure; allowing the system to function entirely offline with no cloud dependencies.

Ollama Local LLM Runtime Engine:

- **Purpose:** To execute the quantized mistral-7B language model locally on the host to manage general, open-domain voice queries.
- **Integration:** The primary python backend communicates with the Ollama service over local API calls whenever the incoming voice command doesn't match the AIML rule engine.

OpenWakeWord / ONNX Runtime:

- **Purpose:** To run the locally-trained "Hey Genie" keyword-spotting model to dictate speech capture without transmitting any voice data for cloud-based wake word detection.
- **Integration:** AudioServer.py loads the exported hey_genie. ONNX model and constantly scores incoming audio frames; a positive detection opens the recording window passed to Faster-Whisper.

Faster-Whisper Speech Recognition Framework:

- **Purpose:** To provide offline Speech-to-Text (STT) transcription of captured PCM audio streams.
- **Integration:** AudioServer.py continuously feeds captured digital audio buffers into the local Faster-Whisper pipeline to produce text transcripts and log-probability confidence scores.

Piper Text-to-Speech (TTS) Engine:

- **Function:** Converts text responses from AIML or the local LLM into audible speech using offline voice models.
- **Integration:** Receives synthesis requests from the backend orchestrator and streams raw PCM audio chunks over TCP to the Speaker ESP32-S3 module for playback.

Fast API Backend Web Framework:

- **Function:** Acts as the central HTTP REST API gateway for sensor telemetry collection and device control endpoints.
- **Integration:** Now handles sensor tasks independently: besides sending GET commands (/relay) to the IoT Relay Board, it runs its own TCP telemetry listener and UDP discovery broadcast, translating Node 1/Node 2 readings into a live sensor state that the AIML layer queries directly (the earlier single-board /update sensor endpoint is no longer used).

Python AIML Engine:

- **Function:** Executes deterministic pattern matching for fast, low-latency intent classification (e.g., relay toggling and sensor status queries).
- **Interaction:** Evaluates compiled XML pattern files (relay. Aiml and sensor. Aiml) against transcribed text input prior to triggering hardware endpoints or defaulting to the LLM.

PySerial (Serial Communication Library):

- **Purpose:** Reads newline-delimited JSON telemetry frames from Node 1 (safety/sensor board) over a direct USB-serial connection at 115200 baud, via a lightweight standalone bridge (serial_bridge.py) that forwards them into api.py without loading the full voice stack.
- **Interaction:** Implemented and unit-tested (51 assertions in the sensor integration test suite, 14 in the end-to-end wire-loop test); Node 1's telemetry is translated into the API's live sensor state by a dedicated bridging module. Confirming this over a real COM port on hardware is the one remaining step, pending resolution of an intermittent power-supply issue on the sensor boards.

Arduino ESP32 Core & Hardware Driver Libraries:

- **Purpose:** Provides low-level board support, Wi-Fi drivers, I2S digital audio protocols, display rendering, ESP-NOW peer-to-peer messaging, and sensor communication protocols (I2C, One Wire).
- **Interaction:** Compiled directly into C++ firmware flash able across all
- ESP32-S3 microcontroller nodes to drive physical peripherals (INMP441, MAX98357A, ILI9341 display, dual MPU-6050, dual MQ-2, dual SW-420, DHT11, and the Node 2 VL53L1X time-of-flight array).

Localized Wireless Access Point (Wi-Fi Router / Hotspot)

- **Purpose:** Sets up the private local area network (SSID: GENIE1) connecting the host laptop and all ESP32-S3 microcontroller nodes.
- **Interaction:** forwards TCP, UDP auto-discovery broadcasts, and HTTP packets between the microphone, speaker, and relay nodes, without the need of an active internet connection. The Node 1/Node 2 safety subsystem is not utilizing this network, as it utilizes ESP-NOW and a direct USB-serial link.2.5

2.5 Related Projects with feature comparisons

2.5.1 Related Projects

Commercial Cloud Voice Assistants (e.g., Amazon Alexa / Google Home)

- **Purpose:** Enable voice-controlled smart home automation, media streaming, and broad open domain question answering via cloud-based processing.
- **Relevance:** Shows strong market acceptance of voice user interfaces, but depends entirely on external cloud infrastructure (serious data privacy concerns, network latency, system failures when the internet is down).

Open-Source Local Smart Home Platforms (e.g., Home Assistant)

- **Purpose:** Consolidate local home automation, device management, and telemetry dashboards across various IoT protocols.
- **Relevance:** Confirms the need for local, privacy-centric home control. However, building complex local AI speech recognition and generative LLM pipelines generally demands expensive home server hardware instead of modular microcontroller-offloaded nodes.

Centralized Microcontroller IoT Systems (e.g., Single-ESP32 Smart Hubs)

- **Purpose:** Connect sensors, small displays, and relays to a single microcontroller board for affordable automation.
- **Relevance:** Emphasizes the cost advantages of ESP32 chips, but encounters major pin conflicts, resource shortages, and memory crashes when trying to run simultaneous digital I2S audio processing, display drivers, and multi sensors polling on a single chip.

2.5.2 Feature Comparison

Table 3: Feature comparison of our product with competitors

Feature / Parameter	Commercial Cloud Assistants (Alexa/Google)	Open-Source Platforms (Home Assistant)	Single-ESP32 Centralized Systems	GENIE System (Our Approach)
Data Privacy & Local Execution	No (Cloud Dependent)	Yes (Local Server)	Yes (Local Hardware)	Yes (100% Offline Local Pipeline)
Hardware Architecture	Proprietary Monolithic Hardware	Centralized PC/SBC Server	Single-Board Microcontroller	Distributed Modular ESP32S3 Nodes
Speech-to-Text (STT) Processing	Cloud-based STT	Cloud or High-End Server STT	Not Supported / Basic Tone	Faster-Whisper (Local CPU/GPU)
Intent Processing Engine	Cloud NLU / Proprietary AI	Rule-based Automations	Basic Hardcoded Rules	Hybrid AIML Kernel + Ollama Mistral-7B
Dual Microphone Audio Selection	Proprietary Cloud Beamforming	External Microphone Hardware	Not Supported	Dual I2S Input Log-Probability Score
Expressive Visual Display	Static GUI / Standard Text	Y Dashboard UI	Minimal / Text LCD	Animated "Robo Eyes" (Parallel ILI9341)
Zero-Configuration Deployment	Y Cloud Account Signings	Manual IP / YAML Setup	Hardcoded IP in Firmware	UDP Auto-Discovery Protocol (Port 9999)
Internet Dependency	Mandatory	Part Optional	No	None (Operates on Local Wi-Fi AP)
Hardware Cost	Moderate to High	Moderate (Requires SBC/PC)	Low	Low (Cost-Effective Modular ESP32s)

2.6 Context Diagram

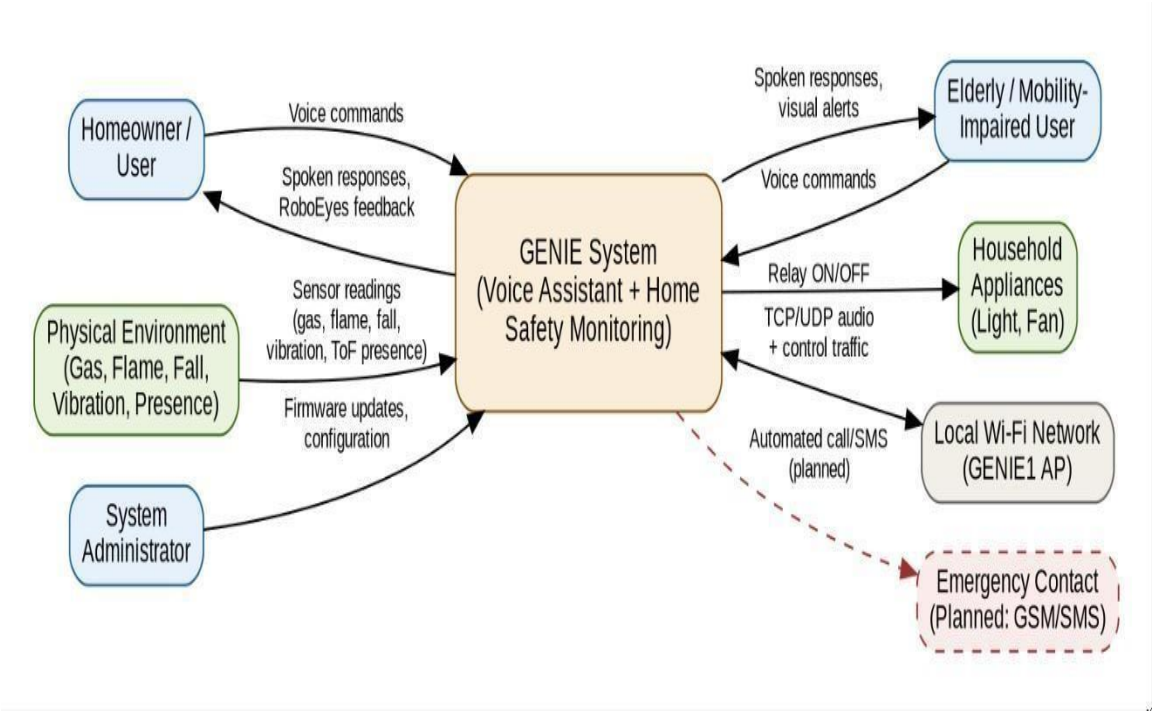


Figure 1: Context Diagram – GENIE System

2.7 Data Flow Diagram Level 0

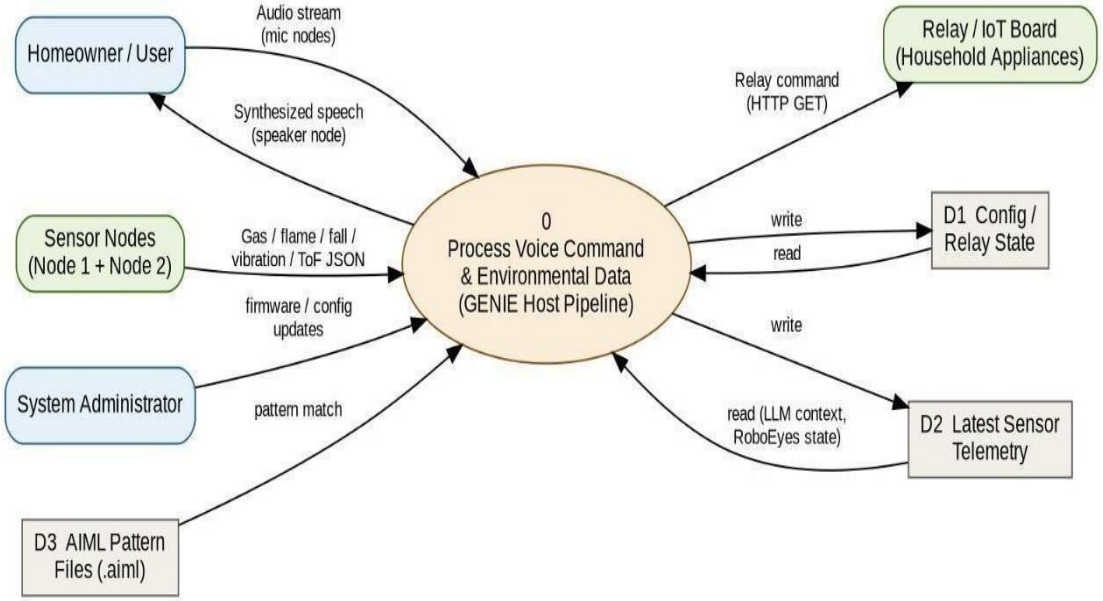


Figure 2: Data Flow Diagram (Level 0) – GENIE System

3. REQUIREMENTS ANALYSIS

3.1 List of Actors

System Boundary Definition

The GENIE boundary of systems consists of all local hardware nodes and software orchestrator services that control interactions with the physical world and perform local AI operations offline. This includes two-microphone audio streaming (AudioServer.py), local voice-activated feature extraction (rms_detector.py), offline speech recognition (Faster-Whisper), local intent classification (AIML pattern kernel and Ollama/Mistral-7B LLM), text-to-speech synthesis (Piper TTS), REST API endpoint (api.py), active-LOW household relay control, sensor telemetry safety processing, dynamic UDP node discovery, and animated display feedback ("Robo Eyes" on ILI9341). External entities interacting with this boundary are primary human users (homeowners, elderly, and physically impaired individuals), physical environmental inputs (smoke, flame, motion/fall spikes, and temperature), high-voltage household loads (lights and fans), and the local Wi-Fi router network (GENIE1).

Homeowner / General User

- **Description:** A typical user who uses GENIE for hands-free voice commands and to ask the system questions.
- **User Story 1:** As a Homeowner, I want to manage household devices like lights and fans using spoken commands so that I can automate my home with ease.
- **User Story 2:** As a Homeowner, I want to pose open-ended questions to an offline AI model so that I can receive answers without risking my data privacy.

Physically Impaired or Elderly User

- **Description:** A person with mobility, motor, or visual challenges who needs accessible hands-free automation and visual feedback.
- **User Story 1:** As a Physically Impaired User, I want to control appliances through voice commands instead of physical switches or mobile apps so that I can live independently.
- **User Story 2:** As an Elderly User, I want to see clear facial expressions ("Robo Eyes") on the system screen so that I can readily understand the device's current state and safety status.

Household Member or Caregiver

- **Description:** A resident or caregiver who depends on continuous safety monitoring to guard the household against domestic dangers.
- **User Story 1:** As a Caregiver, I want the system to keep watching for smoke, fire, and falls so that immediate local visual and audible alerts are triggered in emergencies.
- **User Story 2:** As a Household Member, I want to request live room temperature and humidity readings so that I can verify indoor environmental conditions.

System Maintainer or Installer

- **Description:** The developer or technical user in charge of deploying, flashing, and configuring system hardware and software services.
- **User Story 1:** As a System Maintainer, I want microcontrollers to dynamically find the host server's IP address through UDP auto-discovery so that network setup needs no static IP configuration.
- **User Story 2:** As a System Maintainer, I want all voice recognition and LLM inference to operate strictly offline so that system functions stay available during internet outages.

Safety and Environmental Sensors

- **Description:** Hardware sensors (MQ-2, Flame IR, DHT11, MPU-6050, IR obstacle) that generate physical telemetry.
- **User Story 1:** As a Safety Sensor Node, I want to send POST telemetry updates to the central API every second so that environmental hazards and physical fall events (200 Hz accelerometer spikes) are assessed right away.

Household Relays (Light and Fan)

- **Description:** High-voltage electrical load circuits that are driven by active LOW relay channels.
- **User Story 1:** As an Electrical Appliance Load, I need to get HTTP state commands (/relay) from the host API so that power states toggle reliably after voice intent is confirmed.

3.2 Use Cases

The actors mentioned above interact with GENIE through the use cases described below. Three of the above are unusual for a voice assistant and deserve attention. UC-03, UC-05, and UC-06 do not have a human initiator. They are triggered by the system itself upon a verified hazard, a climate reading above a threshold, or the board determining on its own hardware that it is being managed. These features distinguish GENIE from a device that simply responds when spoken to.

Use Case Summary

UCID	Use Case	Primary Actor	Initiated By	Traces to
UC-01	Control a household appliance by voice	Homeowner / Physically Impaired User	User (spoken command)	EP3-01
UC-02	Ask for a live sensor or climate reading	Homeowner / Caregiver	User (spoken question)	EP2-03, EP3-02
UC-03	Escalate a confirmed emergency without being asked	Household Member (beneficiary)	System (sensor watch)	EP3-02, EP3-03

UC-04	Recognize and react to being handle (edge AI)	Household Member	System (on-board classifier)	EP6-01, EP6-03, EP7-02
UC-05	Run the fan when the room becomes hot or humid	Homeowner (beneficiary)	System (climate watch)	EP7-01
UC-06	Continue a conversation without repeating the wake word	Homeowner	System (follow-up window)	EP7-03
UC-07	Command GENIE remotely by SMS	Remote Caregiver	External (inbound SMS)	EP7-04
UC-08	Acknowledge or redirect an emergency call by keypad	Emergency Contact	External (DTMF digit)	EP3-03, EP7-04
UC-09	Collect data and retrain the on-device model	Developer / Maintainer	Developer	EP6-02
UC-10	Verify the whole pipeline with no hardware attached	Developer / Maintainer	Developer	NFR-05

Table 4: GENIE Use Case Summary

Four of these are fully detailed below: one started by the user, two by the system, and one by an external party.

UC-01 Control a Household Appliance by Voice

Use Case ID	UC-01
Name	Control a household appliance by voice
Primary Actor	Homeowner / Physically Impaired User
Secondary Actors	Relay Node (light, fan)
Description	The user switches a mains appliance without touching a switch, a phone or a screen.
Preconditions	The host pipeline and api.py are running; the mic, speaker and relay nodes have discovered the host over UDP; the relay node is powered.
Trigger	The user says "Hey Genie", then a command such as "turn on the light".

Main Success Scenario	1. The mic node sends interleaved stereo PCM to AudioServer.py. 2. The beamformed channel triggers the wake-word model on three consecutive blocks with a score over 0.65. 3. webrtcvad opens a recording, writes both mic channels to WAV, faster-whisper transcribes both, and takes the higher avg_logprob. 4. Intent Classifier matches the transcript against relay. aiml and returns a command intent.
Alternate Flows	3a. No wake word is detected nothing is recorded and the system remains idle. 4a. Neither channel confidently identifies speech and the clip is discarded rather than passed onward. 5a. No AIML pattern matches and the query is not sensor-related it falls back to the local Mistral-7B model instead. 6a. The relay node is unreachable the failure is reported to the user instead of being silently confirmed.
Postcondition	The appliance is in the requested state and api.py relay state reflects it.
Traces to	EP3-01, EP2-01, EP2-02, EP2-03, NFR-01

UC-03 Escalate a Confirmed Emergency Without Being Asked

Use Case ID	UC-03
Name	Escalate a confirmed emergency without being asked
Primary Actor	Household Member (beneficiary — does not initiate)
Secondary Actors	Node 1 sensors, sensor watch, Modem Node, Emergency Contact
Description	The GENIE detects a hazard, decides for itself that it is real, and gets to a human being. There is no verbal request and the ASR/LLM pipeline is completely bypassed.
Preconditions	Node 1 is reporting telemetry; the modem node holds an open socket to the host; at least one emergency contact is configured.
Trigger	A smoke, flame or fall reading enters an alerting state

Main Success Scenario	1. Node 1 confirms the reading in firmware first three agreeing polls for flame, or a free-fall / impact / stillness sequence for a fall 2. The telemetry line makes it to api.py, where it is stored as sensor state 3. sensor watch. poll once () sees a rising edge, applies the per-sensor debounce (0 s for flame and fall, 4 s for smoke) and confirms the alert 4. A trigger is sent over TCP to the modem node 5. The modem sends an SMS naming what happened, then opens a 45-s window for an acknowledgement 6. With no acknowledgement, it places a voice call to the contact 7. GENIE announces the situation out loud in the room and forces the alarm expression on the face.
Alternate Flows	1. The reading clears before the debounce expires no alert is raised. 2. An alert fired within the last 600s the cooldown suppresses this one, and the channel must go low and high again before it can fire. 3. The sensor reads "unknown" (its self-check flag is false, or the node is silent) "unknown" belongs to no alerting state, so it can neither raise an alarm nor give a false all-clear. 4. The contact replies to the SMS within the window the call is not placed. 5. The host is unreachable the modem node still completes the SMS and call on its own; only the spoken announcement is lost.
Postcondition	A human contact has been informed, exactly once per episode, and the cooldown is armed.
Non-functional notes	Fall and flame reach the host in roughly 50–150 ms because the confirmation happens on the board. Smoke is deliberately slower (≈ 4 s) so a twitching MQ-2 cannot place a phone call.
Traces to	EP3-02, EP3-03, NFR-02, NFR-03, NFR-04

UC-04 Recognize and React to Being Handled (On-Device Edge AI)

Use Case ID	UC-04
Name	Recognize and react to being handled (on-device edge AI)
Primary Actor	Household Member

Secondary Actors	MPU-6050 IMU, on-board MLP classifier, reaction watch, Speaker Node
Description	The face board classifies its own motion on its own processor and remarks on it. This is the only inference in the system that does not happen on the laptop.
Preconditions	har_features.py and a matching har_model.py are on the board; the IMU is answering; the board has passed its start-up warm-up period.
Trigger	Somebody taps, pushes, picks up or shakes the unit.
Main Success Scenario	1. node1 report poll fall () samples the IMU on the same feed the fall detector uses. 2. Each sample is pushed into the Feature Window ring buffer; every 10 samples a 50-sample window closes and yields 16 features. 3. har_model_predict () selects its 14 columns, applies the frozen training standardization and evaluates the 14-12-4 network in pure Micro Python. 4. If the SoftMax confidence is at least 0.70, the class is emitted as an event in the telemetry stream. 4. api.py appends it to an append-only event log with a cursor. 6. reaction watch drains the log, keeps only the newest event, and asks ai. build_reaction_reply () for a line from sensors. Aiml random set. 5. Piper speaks it and the face expression changes.
Alternate Flows	1. The IMU returns nothing the whole window is discarded rather than computed over nonadjacent samples. 2. Confidence is below 0.70 the board says nothing. Measured on the held-out session this raises accuracy on the windows it does answer from 72.6% to 82.6%, at the cost of silence on 22.2% of them.
	3. A fall is active handling events are suppressed entirely; the fall FSM owns that moment. 4. Several events arrive in one poll only the newest is spoken, so "lifted → still lifted → set down" produces one line, not three. 5. GENIE is already speaking the reaction is dropped, never queued.
Postcondition	At most one spoken remark per handling episode; the event cursor has moved forward and cannot replay.

Non-functional notes	Inference is 232 multiply-accumulates against a fixed 800-byte ring buffer, inside the face board's existing 33.3 MS cooperative loop budget. No TensorFlow Lite, no NumPy, no custom firmware build.
Traces to	EP6-01, EP6-03, EP7-02, NFR-08, NFR-09

UC-07 Command GENIE Remotely by SMS

Use Case ID	UC-07
Name	Command GENIE remotely by SMS
Primary Actor	Remote Caregiver
Secondary Actors	Modem Node, host intent pipeline
Description	A caregiver who is not in the house texts a question or a command and receives a written answer, using the same intent engine a spoken command would use.
Preconditions	The modem node is registered on the network with a working SIM; the sender's number is on the allowed list; the host socket is open.
Trigger	An SMS arrives on the modem's SIM.
Main Success Scenario	The modem node polls its inbox from the idle loop only never during an alert, because both readers consume the same unread list. The message and sender are relayed to the host over the dedicated TCP link. The host classifies it exactly as it would a transcript: relay command, sensor question, status query, or LLM fallback. The reply is truncated to the SMS length limit and sent back to the modem node. The modem texts it to the original sender.
Alternate Flows	1a. An alert is in progress inbox polling is suspended, so an acknowledgement reply cannot be mistaken for a command and answered by the LLM instead of cancelling the call. 2a. The host is unreachable the message is not answered, but the node's ability to place an emergency call is unaffected. 3a. The sender is not on the allowed list the message is ignored.
Postcondition	The caregiver has an answer by SMS; no state on the host has been left inconsistent.
Traces to	EP7-04, EP3-03, NFR-01

3.3 Product Backlog

The GENIE Product Backlog arranges the system's development into high-level Epics, detailed Features, User Stories, Acceptance Criteria, and effort estimates based on the Fibonacci Story Point scale.

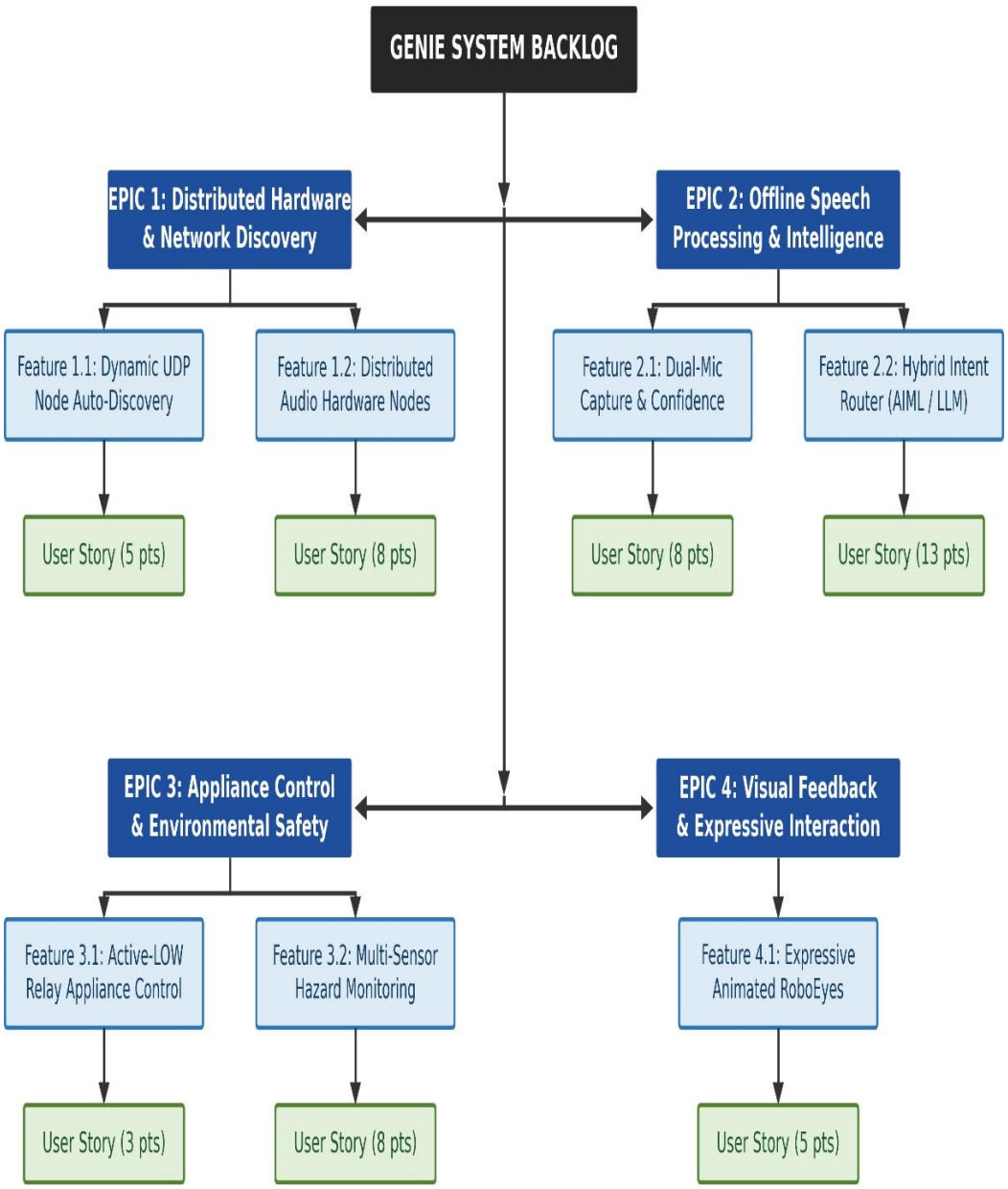


Figure 3: Epic vs User Story in Product Backlog [1]

Epic 1: Distributed Hardware & Network Discovery Infrastructure Feature 1.1: Dynamic UDP Node Auto-Discovery

- **User Story:** In the System Maintainer use case, there is a need for microcontroller nodes to be able to discover the host server's IP automatically via UDP broadcast since any arbitrary network setup is possible and not fixed.

Acceptance Criteria:

- As a System Maintainer, when my ESP32-S3 node boots up on the GENIE1 network and sends out a UDP broadcast message from port 9999, the host laptop replies with its local IP address.
- Once the node receives the host IP and confirms it, it can establish TCP connections on ports 9000 and 9003.
- **Story Points:** 5

Feature 1.2: Distributed Audio Hardware Nodes

- **User Story:** As a Homeowner, I want separate microphone and speaker hardware nodes placed around the room so that audio streaming and speech playback work smoothly without interference from peripheral hardware.

Acceptance Criteria:

- When the Microphone Node is active and speech occurs, two INMP441 MEMS microphones capture and stream dual 16-bit PCM audio channels to the host server.
- When synthesized speech is produced and PCM chunks arrive via TCP at the Speaker Node, the MAX98357A amplifier plays clear audio through the 5W speaker without any clipping.
- **Story Points:** 8

Epic 2: Offline Speech Processing & Intelligence Engine

Feature 2.1: Dual-Microphone Voice Capture & Confidence Routing

- **User Story:** As a User, I want the system to automatically select the clearest microphone signal so that background noise does not interfere with my spoken command.

Acceptance Criteria:

- When dual PCM audio streams are captured and speech stops, rms_detector.py creates timestamped WAV audio pairs (recording_& lt; ts & gt; _A.wav and _B.wav).
- Given two WAV files, after Faster-Whisper transcribes both channels, the selection module compares log-probability confidence scores (avg_logprob) and forwards only the higher confidence transcript to the intent engine.
- **Story Points:** 8

Feature 2.2: Hybrid Intent Router (AIML + Ollama LLM)

- **User Story:** As a User, I want quick execution for simple device commands and thorough conversational answers for general questions, while keeping all processing offline.

Acceptance Criteria:

- When transcribed text input is matched against compiled pattern kernels (relay. aiml or sensor. Aiml), response times stay below 200 milliseconds.
- When an unmatched query is directed to the local Ollama instance that runs quantized Mistral-7B, a contextual text response is produced on-site.
- **Story Points:** 13

Feature 2.2: Hybrid Intent Router (AIML + Ollama LLM)

- **User Story:** As a User, I want quick execution for simple device commands and detailed conversational answers for general questions, with all processing done offline.

Acceptance Criteria:

- Given a transcribed text input, when it is matched against compiled pattern kernels (relay. aiml or sensor. Aiml), response times are consistently under 200 milliseconds.
- When an unmatched query is routed to the local Ollama instance running quantized Mistral-7B, a contextual text response is generated locally with no active internet connection.
- **Story Points:** 13

Epic 3: Appliance Control & Environmental Safety Automation Feature 3.1:

Active-LOW Relay Appliance Switching

- **User Story:** As a Physically Impaired User, I want to control high-voltage lights and fans through natural spoken commands so I can manage my living space on my own.

Acceptance Criteria:

- Given a valid voice command like "turn on the light," when the AIML kernel parses the intent, the host API sends an HTTP GET request (/relay? Light=1) to the relay node.
- When the relay node receives the HTTP command and the signal is asserted LOW, the matching relay channel triggers instantly.
- **Story Points:** 3

Feature 3.2: Multi-Sensor Hazard Monitoring & Telemetry

- **User Story:** As a Caregiver, I want ongoing monitoring for smoke, fire, and physical falls so that household hazards trigger immediate local alerts.

Acceptance Criteria:

- Given Node 1 and Node 2 are operational, when a gas, flame, fall, vibration, or presence reading exceeds its safety threshold, the host's live sensor state (fed by the board telemetry listener) raises a high-priority alert.
- When an active hazard alert is present and evaluated by api.py, immediate visual alert expressions are sent to the display module.
- **Story Points:** 8

Epic 4: Visual Feedback & Expressive Interaction Feature

4.1: Expressive Animated "Robo Eyes" Display

- **User Story:** As an Elderly User, I want to see clear, animated facial expressions on the system screen so that I can visually verify system states and alerts without reading complex text.

Acceptance Criteria:

- Given the system is in standby, listening, processing, or alert mode, when state transitions occur, the parallel ILI9341 display shows animated eye expressions that match the corresponding mood state.
- When a hazard threshold is breached and an alert signal is received, the display immediately replaces current animations with an expressive visual warning state.
- **Story Points:** 5

3.3.1 Backlog Table Epic 1: Distributed Hardware & Network Discovery Infrastructure

PB-ID	User Story	Priority	Acceptance Criteria	Story Points	Status
EP1-01	Fron the perspective of a System Maintainer, I wish for microcontroller nodes to dynamically discover the host server's IP over UDP broadcast so that the system needs zero static network configuration.	High	Given an ESP32-S3 node boots on GENIE1, when it broadcasts on UDP port 9999, then the host responds with its current IP. Given the host IP is received, when the node connects, then it establishes its TCP session(s) with the host.	5	Done

EP1-02	As a Homeowner, I want discrete microphone and speaker hardware nodes so that audio streaming and playback work without peripheral interference.	High	Giventhe Microphone Node is active, when speech occurs, then two INMP441 mics stream dual PCM audio to the host. Given synthesized speech is generated, when PCM chunks arrive at the Speaker Node, then the MAX98357A amplifier plays audio without clipping.	8	Done

3.3.2 Backlog Table — Epic 2: Offline Speech Processing & Intelligence Engine

PB-ID	User Story	Priority	Acceptance Criteria	Story Points	Status
EP2-01	From the perspective if a User, I wanted the system to automatically pick the clearer microphone signal so that background noise does not ruin my spoken command.	High	Given dual PCM audio streams are captured, when speech stops, then paired WAV recordings are produced for both channels. Given two WAV files, when Faster-Whisper transcribes both, then the higher confidence transcript (avg_logprob) is passed to the intent engine.	8	Done
EP2-02	As a User, I want a wake word to gate listening so that GENIE only starts recording when I actually address it.	High	Given ambient audio is streaming, when the trained "Hey Genie" model (Hey_genie. ONNX) scores above threshold, then full speech capture begins. Given no wake word is detected, when ambient conversation	8	Done

			continues, then no recording is triggered.		
EP2-03	As a User, I want fast execution for simple device commands and detailed conversational answers for general questions, while keeping all processing offline.	High	Given a transcribed text input, when matched against compiled AIML patterns (Relay. aiml or sensor. Aiml), then a response is returned deterministically. Given an unmatched, no sensor query, when routed to	13	Done

3.3.3 Backlog Table — Epic 3: Appliance Control & Environmental Safety Automation

PB-ID	User Story	Priority	Acceptance Criteria	Story Points	Status
EP3-01	As a Physically Impaired User, I want to control lights and fans using spoken commands so that I can manage my living space independently.	High	Given a voice command such as "turn on the light", when the AIML kernel parses the intent, then the host issues a Relay cmd. Given the relay node receives the command, when the signal is asserted LOW, then the corresponding channel switches instantly.	3	Done
EP3-02	As a Caregiver, I want continuous monitoring for smoke, fire, falls, vibration	High	Given Node 1 and Node 2 are reporting, when a gas, flame or fall reading crosses	13	Code complete (86 automated assertions passing); hardware

	and presence so that household hazards trigger immediate alerts.		a threshold (confirmed by the fall FSM/gas-ok checks), then the host's sensor state raises the corresponding alert within roughly 50-150 MS for fall/flame.		bring-up blocked by an intermittent sensor board power issue
EP3-03	As a Caregiver, I want GENIE to autonomously call n or text an emergency contact when a critical hazard is confirmed, so that help can be reached even if no one is nearby.	High	Given a confirmed fall, flame or smoke alarm, when sensor watch raises it, then a trigger is sent over a dedicated TCP link to the GSM modem node. Given the modem node receives the trigger, when it executes its call/SMS routine, then an SMS is sent followed by a voice call to the configured contact.	8	Modem firmware and trigger logic implemented Unit pending SIM network registration (PTA/DIRBS) and hardware bring-up

3.3.4 Backlog Table Epic 4: Visual Feedback & Expressive Interaction

PB-ID	User Story	Priority	Acceptance Criteria	Story Points	Status
EP4-01	As an Elderly User, I want to see clear, animated facial expressions on the system screen so that I can visually verify system state without reading text.	Medium	Given the system is in standby, listening, processing or alert mode, when state transitions occur, then the ILI9341 display renders a matching "Robo Eyes" expression. Given a hazard alarm is active, when the alert is raised, then the display overrides its current animation with an alert expression.	5	Laptop to board emotion plumbing implemented; not yet confirmed on real hardware

3.3.5 Epic 5: Remaining Evaluation Gates

PB-ID	User Story	Priority	Acceptance Criteria	Story Points	Status
EP5-01	As a System Administrator, I want a browser-based, sign-in protected dashboard so that I can review system and sensor status remotely.	Medium	Given valid credentials, when a user signs in, then the dashboard displays current relay and sensor state. Given no valid session, when the dashboard is accessed, then access is denied.	8	Not started
EP5-02	As a Homeowner, I want the hardware nodes to run on battery power so that GENIE keeps working through a power interruption.	Medium	Given mains power is unavailable, when a node is running on battery, then it continues normal operation for a defined minimum duration.	5	Not started

3.3.6 Backlog Table Epic 6: On-Device Edge AI

PB-ID	**User Story**	**Priority**	**Acceptance Criteria**	**Story Points**	**Status**
EP6-01	As a Household Member, I want GENIE to work out on the device itself whether it is being tapped, pushed, picked up or shaken, so that it reacts without anything being sent to the laptop.	High	Given the IMU stream, when a 50-sample window closes, then Har features produce a 16value vector and Har model. predict () returns a class and a confidence, entirely in Micro Python on the ESP32-S3 no TensorFlow Lite, no NumPy, no custom firmware. Given a gap in the IMU stream, when a sample is missing, then the whole window is discarded rather than computed over samples that are not adjacent in time.	13	Done

EP6-02	As a Developer, I want the classifier scored on a session it	High	Given har_data.csv holds more than one recording session,	8	Done
	has never seen, so that the accuracy quoted in this report is a real generalization number rather than a memory test.		when har_train.py runs, then it reports both a random-split score and a session-held-out score and names the second as the one to quote. Given the two differ, when the report is written, then the held-out figure is the one used (measured: 88.3% random split vs. 72.6% held out, a 15.7-point gap).		
EP6-03	As a Household Member, I want GENIE to stay silent when it is not sure what just happened, so that it never reacts to the wrong thing.	High	Given a classified window, when the SoftMax confidence is below 0.70, then no event is emitted at all. Given the gate is applied, when measured on the held-out session, then accuracy on the windows it does answer rises from 72.6% to 82.6%, while it stays silent on 22.2% of them.	5	Done

EP6-04	As a Developer, I want the learned model compared against the handwritten rules it replaces, so that the report can say what the model actually bought.	Medium	Given handling_fsm.py already produces the same five events from tuned thresholds, when the MLP is evaluated, then both are reported side by side rather than the model alone. Given a decision tree is also trained, when results are published, then its score is reported too, because a tree can be Printed.	5	Done

3.3.7 Backlog Table Epic 7: Autonomous and Assistive Behaviors

PB-ID	User Story	Priority	Acceptance Criteria	Story Points	Status
EP7-01	As a Homeowner, I want GENIE to run the fan itself when the room gets hot or humid and to tell me why, so that I do not have to ask.	Medium	Given a temperature at or above 32 °C or humidity at or above 70%, when climate watch polls, then the fan relay is switched on and a spoken line names the reading that caused it. Given I say "turn off the fan", when the rule next evaluates, then my override wins and keeps winning until the situation band itself changes. Given the DHT11 stops answering, when the reading is null, then the previous decision is held rather than	8	Done
EP7-02	As a Household Member, I want GENIE to remark when it is pushed or picked up, so that it reads as aware of its own situation rather than as an instrument that	Medium	Given the board reports a handling event, when reaction watch drains api.py event log, then the newest event only is spoken, drawn from a random set so the same sentence is not repeated. Given GENIE is already speaking, when	8	Done

	only answers questions.		a reaction falls due, then it is dropped rather than queued. Given the laptop has just started, when the first poll runs, then the cursor jumps to the latest event rather than replaying a backlog.		
EP7-03	As a Homeowner, I want to ask a follow-up question without saying "Hey Genie" again, so that talking to GENIE feels like a conversation.	Medium	Given GENIE has finished speaking, when the eight second follow-up window is open, then the voice activity detector alone may open a recording. Given the window expires, or three follow-ups have run back-to-back, then the wake word is required again. Given unclear audio inside the window, when nothing intelligible is transcribed, then GENIE stays silent so the window can close instead of prompting itself into a loop.	8	Done
EP7-04	As a Remote Caregiver, I want to text GENIE and get an answer, and to clear an emergency call from the keypad, so that I can act on an alert without being in the house.	Medium	Given an SMS from an allowed number, when the modem node relays it, then the host answers it through the same intent pipeline a spoken command uses and the reply is texted back. Given an emergency call is answered, when the contact presses 1, then the escalation stops; pressing 2 moves it to the next contact.	13	Partial firmware and host logic implemented and unit tested;

3.3.8 Non-Functional Requirements

PB-ID	User Story	Priority	Acceptance Criteria	Story Points	Status
NFR-01	As a User, I want the entire voice and safety pipeline to work with no internet connection so that my privacy is preserved and outages don't disable safety monitoring.	High	Speech recognition (Whisper), intent handling (AIML/Mistral via Ollama) and speech synthesis (Piper) all run on the local host. No household audio or sensor telemetry is transmitted outside the local network.	-	Done
NFR-02	As a Caregiver, I want fall and flame alarms to reach GENIE within a fraction of a second so that a genuine emergency is never delayed by AI processing time.	High	Given a confirmed fall or flame event, when Node 1's own FSM/threshold logic confirms it, then the alert reaches the host's sensor state in roughly 50-150 ms, independent of Whisper/LLM latency.	-	Done
NFR-03	As a Caregiver, I want smoke and motion/presence readings to be debounced so that a twitchy sensor doesn't wake me with false alarms at night.	High	Smoke alerts require a deliberate ~4 s debounce on the MQ-2 reading. Motion/presence alerts require a deliberate ~6 s debounce across the ToF array and IMU.	-	Done
NFR-04	As a System Maintainer, I want a sensor channel that goes silent to be clearly distinguished from one that reads safe, so a fault is never mistaken for an all-clear.	High	A disconnected or faulty sensor reports "unknown", which can never satisfy an alarm condition and can never produce a false "all clear" response.	-	Done

NFR-05	As a System Maintainer, I want the core sensor-to-API integration logic covered by automated tests so that regressions are caught before hardware time is spent.	Medium	86 assertions across four automated test files (node bridge self-tests, sensor integration, wire loop, Wi-Fi uplink) pass without any physical board attached.	-	Done
NFR-06	As a System Maintainer, I want Wi-Fi connected nodes to join the network with zero manual configuration so that deployment doesn't require static IP setup.	Medium	A node broadcasting on UDP port 9999 receives the host's IP automatically and connects without any hardcoded address.	-	Done
NFR-07	As a Homeowner, I want the hardware nodes to keep running on battery power during a mains outage.	Medium	Untethered battery operation for a defined minimum duration.	-	Not started
NFR08	As a System Maintainer, I want on-device inference to fit inside the face board’s existing loop budget, so that adding intelligence does not cost the display its frame rate.	High	Inference is a 14→12→4 evaluation of 232 stored numbers in pure Micro Python, over a fixed 50-sample ring buffer of about 800 bytes, with no dynamic allocation per window and no external inference runtime.	–	Done

NFR09	As a System Maintainer, I want the board to refuse a model that does not match its feature extractor, so that a silent train/serve mismatch cannot be mistaken for a false positive.	High	har_model.py compares its stored FEATURE_VERSION against Har features import time and raises rather than loading. The extractor is one shared f imported by both the board and the trainer, so the two cannot compute features differently.	–	Done

NFR10	Being a Homeowner, I want an autonomous action to be explainable and overridable, so that the system never changes my room without saying why or without letting me stop it.	Medium	Every autonomous relay change is accompanied by a spoken line naming the reading that caused it. A spoken override outranks the rule and holds until the situation band itself changes, rather than expiring on a timer.	–	Done

4. PROJECT PLANNING AND EXECUTION USING SPRINTS

Work was divided according to the subsystem instead of the sprint. This allowed every member to take ownership of a cohesive part of the system from beginning to end and be accountable for its acceptance criteria. The following breakdown was applied to the whole project; task allocations for each sprint were determined based on it.

Table 5: Division of Work by Team Member

Member	Roll Number	Primary Responsibility	Approx. Hours
Arman Adil Mangat	F2022376015	Sensor bring-up and testing; delivery of integration-ready sensor modules; AIML pattern authoring; design and fabrication of the GENIE enclosure and body.	~240
Muhammad Suhaib Mamoon	F2022376033	Microphone-to-speaker pipeline; wake word, capture and transcription; local language-model integration and AI inference; Fast API backend; TCP Audio Server management; integration of every module in the speech pipeline; sensor use cases.	~240
M. Farhal Raza	F2022376002	IoT relay box: firmware, wiring and delivery of integration ready files for the appliance-control node.	~180
Muhammad Huzaifa Furqan	F2022376030	Project documentation: report drafting, meeting minutes, backlog and test-case records.	~150
Suhaib and Farhal (jointly)	—	SIM com A7670E cellular emergency module: driver, firmware, wiring and testing, carried out in equal measure by both members.	included above

4.1 Jira

The project's backlog, sprint boards, and task-related assignments are all tracked and managed in Jira. Each Product Backlog item identified in Chapter 3 has been allocated a Unique Jira issue key in the format PB-ID. This key links a backlog item, a sprint task, and a Jira ticket.

4.2 Slack

Daily team communication, stand-up notes, and supervisor coordination take place in a dedicated Slack workspace, with a separate channel for each subsystem (audio pipeline, sensors and edge AI, emergency module, documentation).

4.3 GitHub Repository

The source code is version-controlled on GitHub, distributed over two repositories one containing the laptop-side Python code (AudioServer.py, api.py, and Services package) and the other the ESP32-S3 nodes' microcontroller firmware.

4.4 Sprint 1

The purpose of the first sprint is to set up distributed hardware foundations and get the offline speech pipeline from discovery to final stages: automatic node finding, dual microphone recording with wake word capture, offline speech to text, and reliable intent routing leading to relay activation. These are the most technically challenging and risky features in the backlog, upon which everything else depends.

4.4.1 Sprint 1 Planning Meeting Minutes

Project Name: GENIE Generate, Engage, Navigate Intelligent Environments **Sprint Name:** Sprint 1 Distributed Hardware Foundation & Offline Speech Path **Sprint Duration:** 3 November 2025 to 19 December 2025 (7 weeks)

Meeting Date: 31 October 2025

Facilitated by: FYP Supervisor / Product Owner

Attendees: Arman Adil Mangat, Muhammad Suhaib Mamoon, M. Farhal Raza, Muhammad Huzaifa Furqan.

Backlog things with relatively bigger priority, extreme risk and architectural significance. The following features are extracted from the Product Backlog in Chapter 3:

Table 6: Selected Features of Product Backlog

PB-ID	User Story	Priority	Acceptance Criteria	Story Points	Status
EP1-01	Being a System Maintainer, I want microcontroller nodes to discover the host server's IP over UDP broadcast so that the system needs zero static network configuration.	High	A node broadcasting on UDP port 9999 receives the host IP and connects on the TCP audio and control ports.	5	Done

EP1-02	As a Homeowner, I want discrete microphone and speaker hardware nodes so that audio streaming and playback work without peripheral interference.	High	Two INMP441 microphones stream dual PCM audio to the host; synthesized speech plays through the MAX98357A amplifier without clipping.	8	Done
EP2-01	As a User, I want the system to automatically pick the clearer microphone signal so that background noise does not ruin my spoken command.	High	Both channels are transcribed and the higher avg_logprob transcription is selected.	8	Done
EP2-02	As a User, I want a wake word to gate listening so that GENIE	High	The trained “Hey Genie” model scores above threshold for the	8	Done
	only starts recording when I actually address it.		required number of consecutive blocks before capture begins.		
EP2-03	As a User, I want fast execution for simple device commands and detailed conversational answers for general questions, while keeping all processing offline.	High	AIML patterns resolve device commands; unmatched non-sensor queries fall back to the local Mistral7B model.	13	Done
EP3-01	As a Physically Impaired User, I want to control lights and fans using spoken commands so that I can manage my living space independently.	High	A recognized command issues a relay instruction and the corresponding active LOW channel switches.	3	Done

Action Items before Sprint Starts:

- Confirm GitHub repository and branch strategy Muhammad Suhaib Mamoon
- Flash and bench-verify each ESP32-S3 node Arman Adil Mangat
- Create and assign Jira tasks against PB-IDs Muhammad Huzaifa Furqan
- Set up the dedicated GENIE1 wireless access point M. Farhal Raza

SP-ID	Task Name	Related PB-ID	Assigned To	Status	Est. (h)	Actual (h)	Comments
S1-1	Implement UDP discovery beacon on the host	EP1-01	Muhammad Suhaib Mamoon	Done	5	6	Beacon on UDP 9999; nodes bind with no configured address.
S1-2	Implement node side discovery client and TCP reconnect	EP1-01	Muhammad Suhaib Mamoon	Done	6	8	Reconnect logic needed a second pass after socket drops.
S1-3	Bring up dual INMP441 IZS capture on the microphone node	EP1-02	Arman Adil Mangat	Done	8	10	Both microphones mounted in the enclosure at 10 cm spacing.
S1-4	Bring up MAX98357A playback on the speaker node	EP1-02	Muhammad Suhaib Mamoon	Done	6	7	GAIN pin tied to GND for 12 dB after level testing.
S1-5	Implement deinterleave, high-pass filter and gain stage	EP2-01	Muhammad Suhaib Mamoon	Done	8	10	High-pass applied before gain; deinterleaving done once.
S1-6	Implement paired dual-channel WAV writing	EP2-01	Muhammad Suhaib Mamoon	Done	5	5	Two mono WAVs per capture; the beam channel is

S1-7	Integrate Faster Whisper and confidence-based channel selection	EP2-01	Muhammad Suhaib Mamoon	Done	14	Channel chosen on transcription confidence, not on level.	
S1-8	Record “Hey Genie” Wake word	EP2-02	Arman Adil Mangat	Done	12	Corpus recorded across five speakers and curated.	
S1-9	Train and expo wake-word mod ONNX	EP2-02	Muhammad Suhaib Mamoon	Done	13	Model exported to ONNX and then frozen for the project.	
S1-10	Implement wake-word scoring, threshold and run-length gate	EP2-02	Muhammad Suhaib Mamoon	Done	14	Run-length gate added; threshold held at 0.65.	
S1-11	Implement VAD ca window and silence termination files	EP2-02	Muhammad Suhaib Mamoon	Done	11	RMS floor retuned for the detection channel.	
S1-12	Author relay.aiml sensor.Aiml pattern files	EP2-03	Arman Adil Mangat	Done	11	relay.aiml and sensor.Aiml authored and unit tested.	

S1-13	Implement determine first intent routing LLM fallback	EP2-03	Muhammad Suhaib Mamoon	Done	12	Sensor questions deliberately excluded from the model fallback.	
S1-14	Integrate Piper TT speaker streaming files	EP2-03	Muhammad Suhaib Mamoon	Done	9	Output wave shaping tuned after a distortion	

Table 7: Team Capacity Sprint 1

Member	**Days Available**	**Hours/Day**	**Total Hours**
Muhammad Suhaib Mamoon	32	3.5	112
Arman Adil Mangat	25	3.0	75
M. Farhal Raza	22	2.5	55
Muhammad Huzaifa Furqan	19	2.5	48

4.4.2 Sprint 1 Backlog

The following tasks are decomposed from the Product Backlog features selected above.

Table 8: Sprint 1 Backlog

S1-15	Implement the relay control endpoint and active-LOW node firmware	EP3-01	M. Farhal Raza	Done	24	26	Active-LOW relay board wired; endpoint verified end to end.
S1-16	Document Sprint 1: minutes, backlog and test cases	N/A	Muhammad Huzaifa Furqan	Done	24	31	Sprint 1 minutes, backlog and test-case documentation.

4.4.3 Sprint 1 Design Class Diagram

The class diagram below includes the classes introduced by the features chosen for this sprint: the audio capture and wake-word path, the transcription and confidence-selection stage, and the intent router.

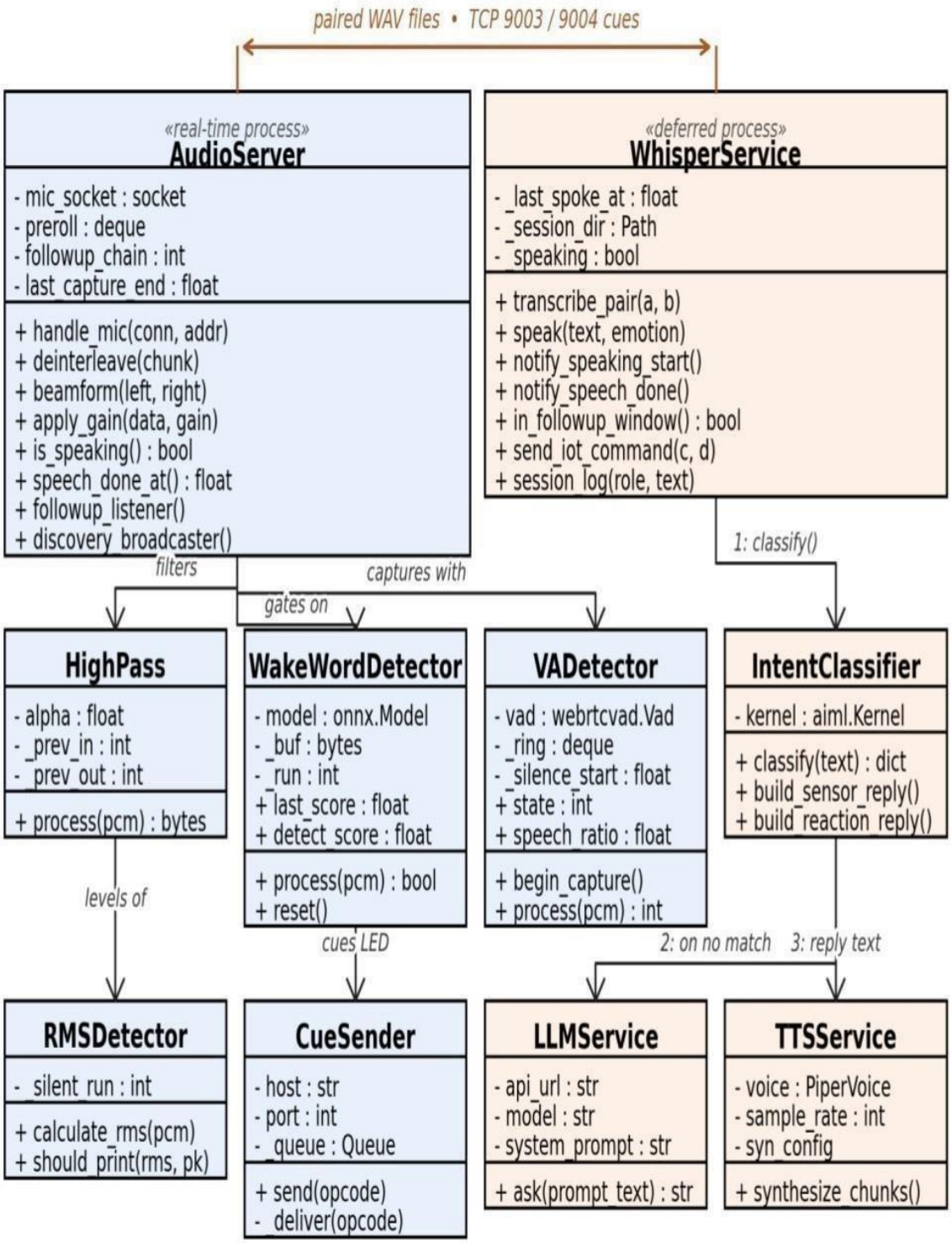


Figure 4: Class Diagram of Sprint 1 Features

4.4.4 Sprint 1 Sequence Diagram

The sequence diagram below outlines the user stories completed in this sprint. The main flow goes from a spoken wake phrase through capture, transcription, intent resolution, and relay actuation, and then back to the spoken confirmation.

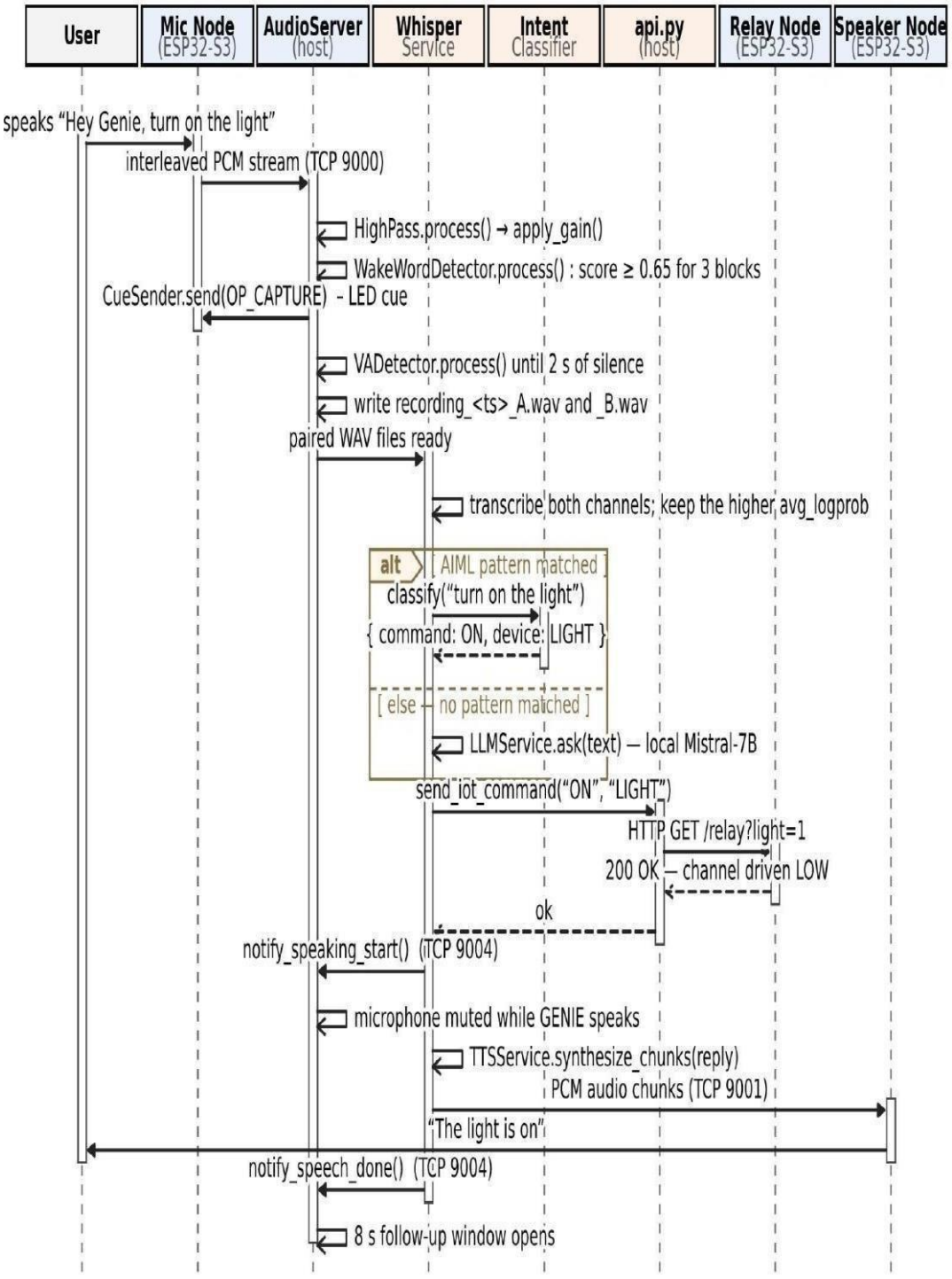


Figure 5: Sprint 1 Sequence Diagram (Wake Word to Relay Actuation)

4.4.5 Sprint 1 Decision Table

A software system is often controlled by a set of conditions yielding different outcomes; a decision table encodes that logic completely. The next table decides whether GENIE opens a recording window for a specific segment of audio. The conditions are:

- Whether the wake-word model achieved a score above the threshold for the required consecutive blocks
- Whether the voice-activity detector indicates continuous speech above the RMS floor
- Whether the system is in the middle of speaking a response (microphone disabled)
- Whether the audio is within an open follow-up window from an earlier response

Table 9: Sprint 1 Decision Table — Recording Window Activation

Condition / Rule	**R1**	**R2**	**R 3**	**R 4**	**R 5**
Wake word confirmed	T	F	F	F	T
Sustained speech detected	T	T	F	T	T
System currently speaking	F	F	F	F	T
Inside follow-up window	–	T	T	F	–
Action: open recording window	Yes	Yes	No	No	No

4.4.6 Sprint 1 Extended Test Cases

A distinct, more detailed test case is written for every user story delivered in the sprint. The following example addresses EP2-02, wake-word-gated capture.

Table 10: Sprint 1 Extended Test Case

Field	**Value**
Test Case ID	TID-S1-01
Test Designed By	Muhammad Suhaib Mamoon
Epic Name and Backlog ID	Offline Speech Processing — EP2-02
Test Designed Date	24-11-2025
Test Priority	High

Test Executed By	Arman Adil Mangat
Test Title	Verify wake-word-gated recording at conversational distance
Test Execution Date	05-12-2025
Description	Confirm that the trained “Hey Genie” model opens a recording window when the wake phrase is spoken, and does not open one for partial phrases or ambient conversation.
Dependencies	Microphone node streaming; hey_genie.onnx loaded; host pipeline running.
Post Condition	Captured audio is written as a paired dual-channel WAV set and passed to the transcription stage.

Step	Test Step	Test Data	Expected Result	Actual Result	Status
1	Position the speaker at 1 m, on-axis to the microphone node.	Distance: 1 m	Node is streaming; system is idle in WAIT_WAKE.	Node streaming; system idle in WAIT_WAKE.	Pass
2	Speak the full wake phrase at a normal conversational level.	“Hey Genie”	Score exceeds 0.65 for the required consecutive blocks; a recording window opens.	Score 0.94; capture opened on the third block.	Pass
3	Speak a command immediately after the wake phrase.	“Turn on the light”	Speech is captured and the window closes after the silence period.	Command captured; window closed after the silence period.	Pass
4	Speak only a partial phrase.	“Genie”	Score remains below threshold; no recording window opens.	Score peaked at 0.57; no capture opened.	Pass
5	Hold ordinary conversation in the room without the wake phrase.	Unrelated speech, 3 minutes	No recording window opens.	No capture opened over three minutes.	Pass
6	Repeat step 2 at 2 m and at 3 m.	Distance: 2 m, 3 m	Recorded at each distance.	9/10 at 2 m, 6/10 at 3 m; recorded in Chapter 8.	Pass

4.4.7 Sprint 1 Review Meeting

Sprint: Sprint 1 – Distributed Hardware Foundation & Offline Speech Path Review Date

19 December 2025

Duration: 1 hour

Facilitator: Product Owner / FYP Supervisor

Attendees: Arman Adil Mangat, Muhammad Suhaib Mamoon, M. Farhal Raza, Muhammad Huzaifa Furqan

The table provides the report on the tasks given to team members in the sprint planning meeting.

Table 11: Status of Sprint 1 Backlog

SP-ID	Task Name	Related PB-ID	Owner	Status	Notes
S1-1	Implement UDP discovery beacon on the host	EP1-01	Muhammad Suhaib Mamoon	Completed	Beacon on UDP 9999; nodes bind with no configured address.
S1-2	Implement node-side discovery client and TCP reconnect	EP1-01	Muhammad Suhaib Mamoon	Completed	Reconnect logic needed a second pass after socket drops.
S1-3	Bring up dual INMP441 IZS capture on the microphone node	EP1-02	Arman at Adil Mang	Completed	Both microphones mounted in the enclosure at 10 cm spacing.
S1-4	Bring up MAX98357A playback on the speaker node	EP1-02	Muhammad Suhaib Mamoon	Completed	GAIN pin tied to GND for 12 dB after level testing.
S1-5	Implement deinterleave, high-pass filter and gain stage	EP2-01	Muhammad Suhaib Mamoon	Completed	High-pass applied before gain; deinterleaving done once.
S1-6	Implement paired dualchannel WAV writing	EP2-01	Muhammad Suhaib Mamoon	Completed	Two mono WAVs per capture; the beam channel is never written.

S1-7	Integrate Faster-Whisper and confidence-based channel selection	EP2-01	Muhammad Suhaib Mamoon	Completed	Channel chosen on transcription confidence, not on level.
S1-8	Record and curate the “Hey Genie” wake word corpus	EP2-02	Arman Adil Mangat	Completed	Corpus recorded across five speakers and curated.
S1-9	Train and export the wake word model to ONNX	EP2-02	Muhammad Suhaib Mamoon	Completed	Model exported to ONNX and then frozen for the project.
S1-10	Implement wake-word scoring, threshold and run length gate	EP2-02	Muhammad Suhaib Mamoon	Completed	Run-length gate added; threshold held at 0.65.
S1-11	Implement VAD capture window and silence-based termination	EP2-02	Muhammad Suhaib Mamoon	Completed	RMS floor re-tuned for the detection channel.
S1-12	Author relay. aiml and sensor. Aiml pattern files	EP2-03	Arman Adil Mangat	Completed	relay. aiml and sensor. Aiml authored and unit-tested.
S1-13	Implement deterministic first intent routing with LLM fallback	EP2-03	Muhammad Suhaib Mamoon	Completed	Sensor questions deliberately excluded from the model fallback.
S1-14	Integrate Piper TTS and speaker streaming	EP2-03	Muhammad Suhaib Mamoon	Completed	Output wave shaping tuned after a distortion measurement.
S1-15	Implement the relay control endpoint and active-LOW node firmware	EP3-01	M. Farhal Raza	Completed	Active-LOW relay board wired; endpoint verified end to end.
S1-16	Document Sprint 1: minutes, backlog and test cases	N/A	Muhammad Huzaifa Furqan	Completed	Sprint 1 minutes, backlog and testcase documentation

Demo Highlights:

- **Zero-configuration discovery:** a node powered on with no configured address joins and begins streaming.
- **Wake-word-gated capture:** "Hey Genie" opens a recording window; ambient conversation does not.
- **Offline round trip:** a spoken command is transcribed, routed and answered aloud with the network cable unplugged.
- **Relay actuation:** a spoken command switches a mains appliance.

Sprint Goal Achievement: The sprint goal was achieved by delivering on the six selected backlog items and demonstrating end to end that a spoken wake phrase opens a capture, command is transcribed offline, intent is resolved deterministically, a mains appliance switches, and a reply is spoken aloud with the network disconnected throughout.

Feedback from Stakeholders:

Feedback Source	Comments
FYP Supervisor	Asked that every performance claim be supported by a measurement rather than a demonstration, and requested an on-device AI component for the following sprint.
Internal QA	Wake-word false activations in a room with conversation were judged too frequent for a device meant to sit idle, and were raised as the main outstanding quality risk.
Team Feedback	The split into two processes made the audio path far easier to debug than expected; the cross-process seams were where the hardest defects turned out to hide.

Metrics Overview:

Metric	Value
Committed Story Points	45
Completed Story Points	45
Carryover Items	0
Bugs Reported / Resolved	7 / 7

What Went Well:

- Splitting the audio path into two operating-system processes proved to be a good decision, as a slow transcription or language-model call would not interfere with microphone capture – which is what the split was for.
- Zero-configuration discovery worked from the first integration test, and it removed the need to type in addresses for every subsequent bench session.
- Making the intent route decision deterministically before querying the language model allowed for speedy tests for device commands and made the results predictable enough to reason about.
- Recording the numbers alongside the thresholds in the configuration file made later adjustments easier to reason about – as opposed to trying to remember what numbers corresponded to what changes in behavior at.

What Could Be Improved:

- Detection thresholds were calibrated on a single microphone and applied to the averaged channel, which silently made them more stringent and interrupted commands mid-sentence. Whenever the signal a threshold applies to changes, that threshold should be recalibrated.
- Two components were allowed to mutate the same shared state, so one component correctly reporting that it had no data wiped out live readings written by the other. Shared state needs to have a clear policy about who is allowed to write nothing.
- The firmware on the boards was not always in sync with the repository source, and appeared like a logic bug and not an outdated deployment.
- Test coverage of the cross-process seams was sparse; both hardest defects were only discoverable when the two processes were looked at together.

Action Items / Next Steps:

Action Item	**Owner**	**Due Date**
Log the confidence score of every false wake-word activation so marginal and high-confidence false accepts can be told apart	Muhammad Suhaib Mamoon	09-01-2026
Re-measure detection rate at 1–2 m now that the run length gate has been raised from two blocks to three	Arman Adil Mangat	09-01-2026
Fit the gas-sensor divider network and re-capture the baseline after a 24–48-hour burn-in	Arman Adil Mangat	16-01-2026

Wrap-up: The team moves into the next sprint focused on the safety and sensor subsystem: hazard monitoring, on-device activity classification and the emergency alert path. The date of the next planning meeting is 9 January 2026.

4.4.8 Sprint 1 Retrospective Meeting

Sprint: Sprint 1 Distributed Hardware Foundation & Offline Speech Path

Retrospective Date: 19 December 2025

Duration: 1 hour

Facilitator: Product Owner / FYP Supervisor
Attendees: Arman Adil Mangat, Muhammad Suhaib Mamoon, M. Farhal Raza, Muhammad Huzaifa Furqan

Sprint Overview. Goal: Establish the distributed hardware foundation and a complete offline speech path from wake word to spoken reply.

Sprint Duration: 3 November 2025 – 19 December 2025 (7 weeks).

What Went Well:

- The team actually measured before declaring victory. The dual microphone beamformer was tested for detection range and false accept rate, not assumed to be a benefit, and the defended result, though negative, was still a defensible position.
- Feature extraction from a single shared file between host trainer and device firmware eliminated train/serv skew by construction, and the two implementations were shown to be equal for every vector.
- Failure modes were designed to conservatively degrade: follow up windows closed rather than follow up microphones opened, and sensor failure resulted in unknown rather than clear.
- Small sample statistics were reported with confidence intervals, avoiding the temptation to read meaning into two samples that had equal, if imprecise, means. What went wrong:
- Infrastructure failures dominated the defect list, including blocked firewall rules inbound and a host address that changed between sessions, both of which hid hardware failures.
- Two results had to be retracted due to a matched control run that contradicted an earlier analysis.
- A test passed even though the deployed code failed because the test used a dependency the module never imported.
- Numerical tolerances were selected based on whichever machine ran the test first, resulting in a test passing on one and failing on the other for no reason besides differing floating point precision. Suggestions:
- Write the measurement that defines each threshold next to the threshold, and retest whenever that threshold is crossed.
- Keep a single source of truth for pin definitions, with code checking against it, not the other way around.
- Test the code under test and only replace dependencies that are necessary to bring the code under test in a reasonable environment. A missing import is not an acceptable replacement.

- Use the precision of the sensor as a baseline for numerical tolerances.
- Have a preflight check that checks that the configuration matches the deployment, or at least has been updated, before running any hardware checks.

Action Items:

Action Item	Owner	Due Date
Add the pre-flight configuration and deployment check to the standard hardware test routine	Muhammad Suhaib Mamoon	09-01-2026
Move fall detection off the display loop so it can meet its 20 MS sample rate	Arman Adil Mangat	23-01-2026
Fix the host address so node discovery is not chasing a moving target between sessions	M. Farhal Raza	09-01-2026

Team Health Check:

- **Team morale:** Positive the sprint goal was achieved and the pipeline worked end to end.
- **Communication:** Effective. Daily coordination happened on Slack, with a weekly review alongside the supervisor.
- **Tools and resources:** Adequate, though shared hardware access limited the amount of parallel testing possible.

Wrap-up: The team is pleased with the outcome of the sprint and agrees that the improvements noted above are appropriate. The recurring lesson learned seems to be that on hardware this constrained, the observed failure is rarely in the layer being examined, and the cheapest next step is usually to isolate the variable rather than fix the symptom.

5. SYSTEM ARCHITECTURE

This chapter outlines GENIE's architecture through the C4 model, starting with a single box system perspective and moving down to the crucial classes within its sensor pipeline.

5.1 System context Diagram

The diagram below depicts GENIE as one central software system, along with the users and external systems it interacts with (C4 Level 1).

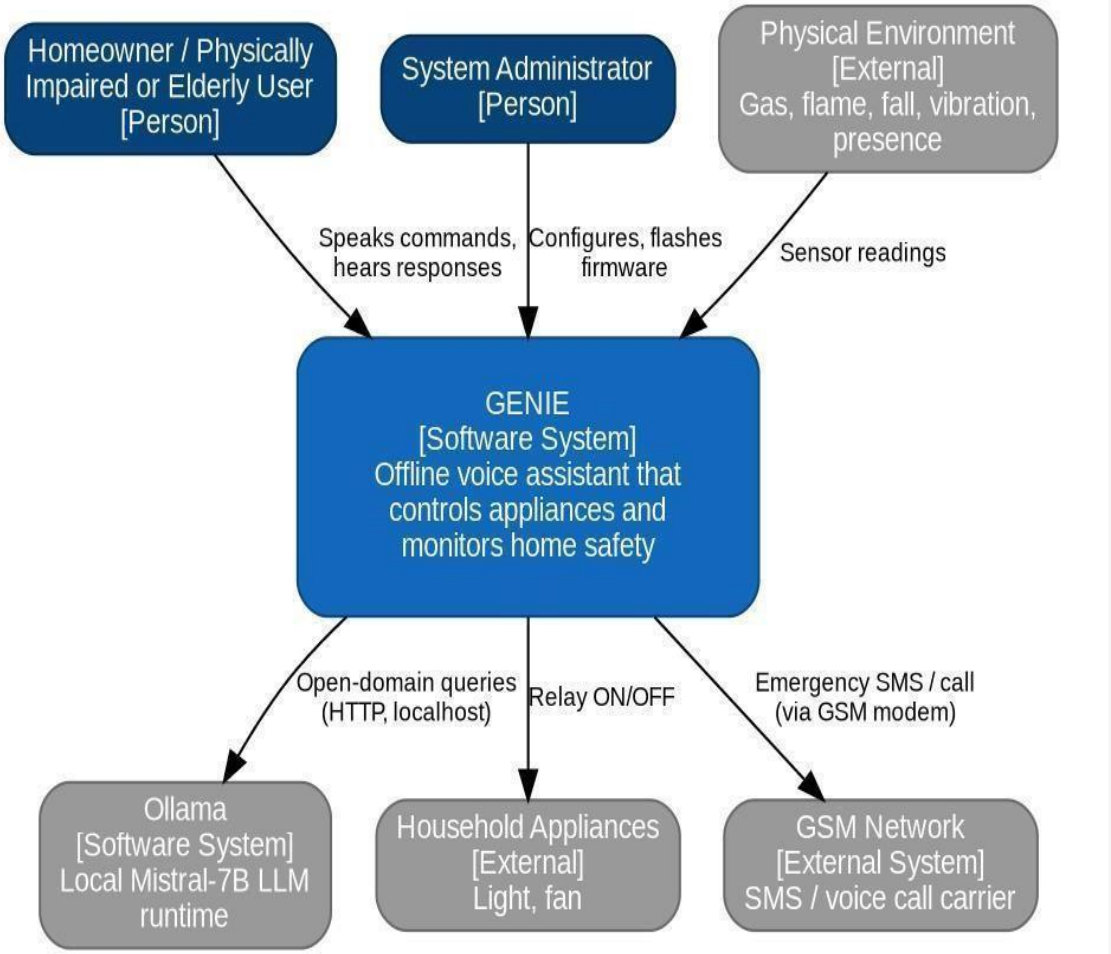


Figure 6: System Context Diagram (C4) — GENIE

5.1 System Container Diagram

The diagram below decomposes GENIE into its deployable components the six ESP32class boards and the laptop host process group along with the protocol used by each connection (C4 Level 2).

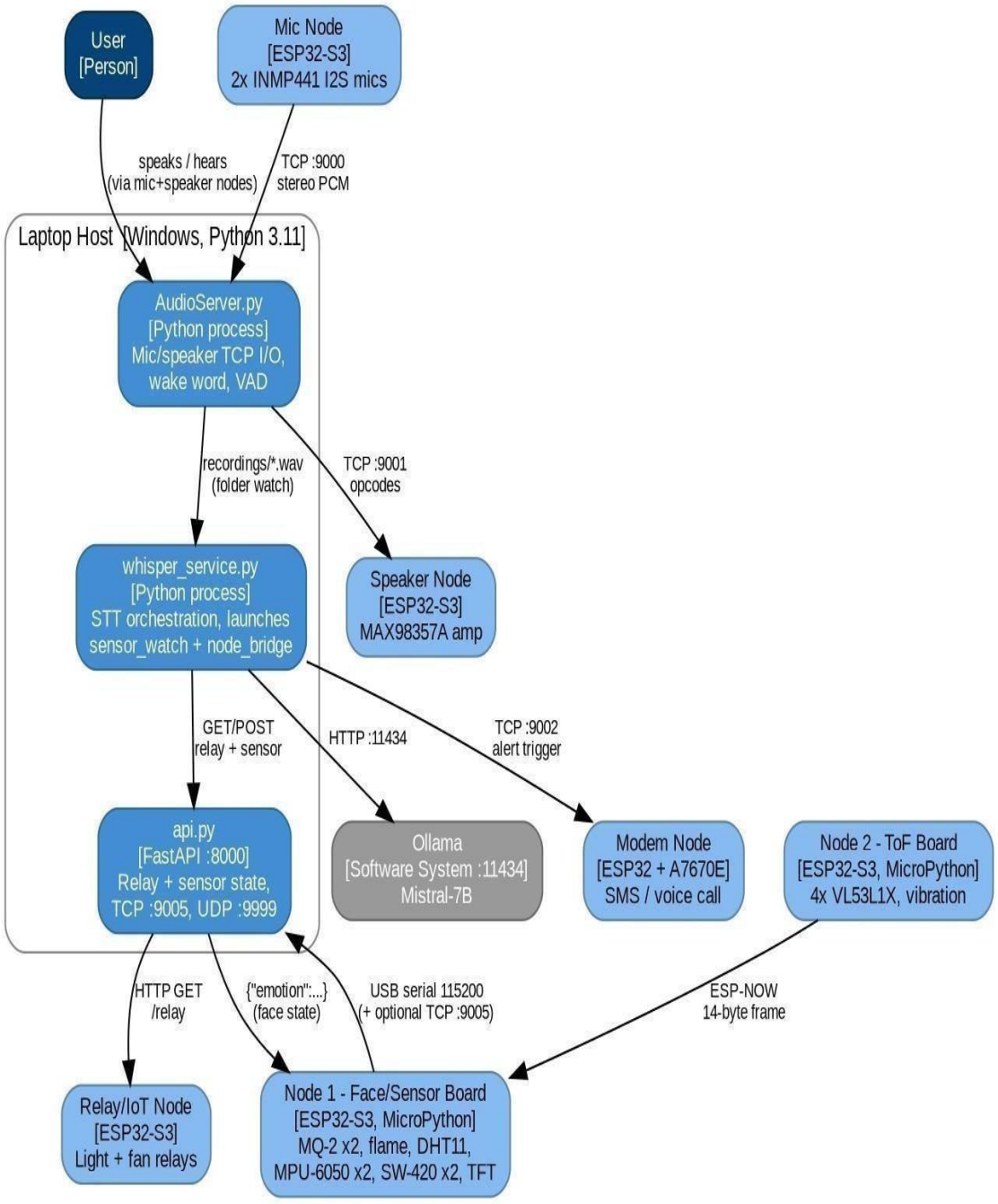


Figure 7: System Container Diagram (C4) — GENIE

5.2 Component Diagram

The diagram below focuses on the Laptop Host container from the previous diagram, revealing its internal parts and their interactions (C4 Level 3).

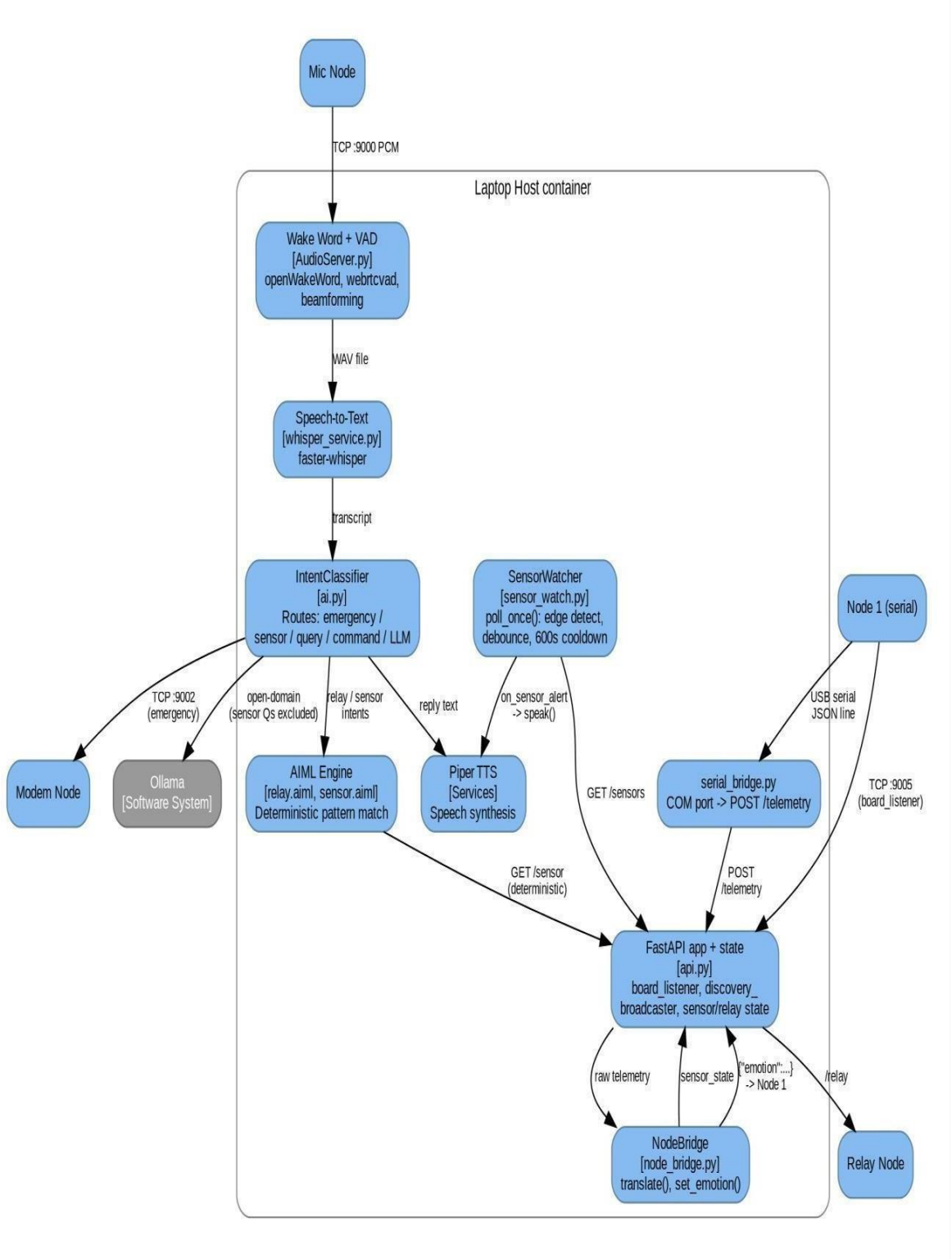


Figure 8: Component Diagram (C4) — GENIE Laptop Host

5.3 Code Diagrams

The following diagram shows the key classes involved in the sensor pipeline specifically, from Node 2's readings through to a spoken alert (C4 Level 4).

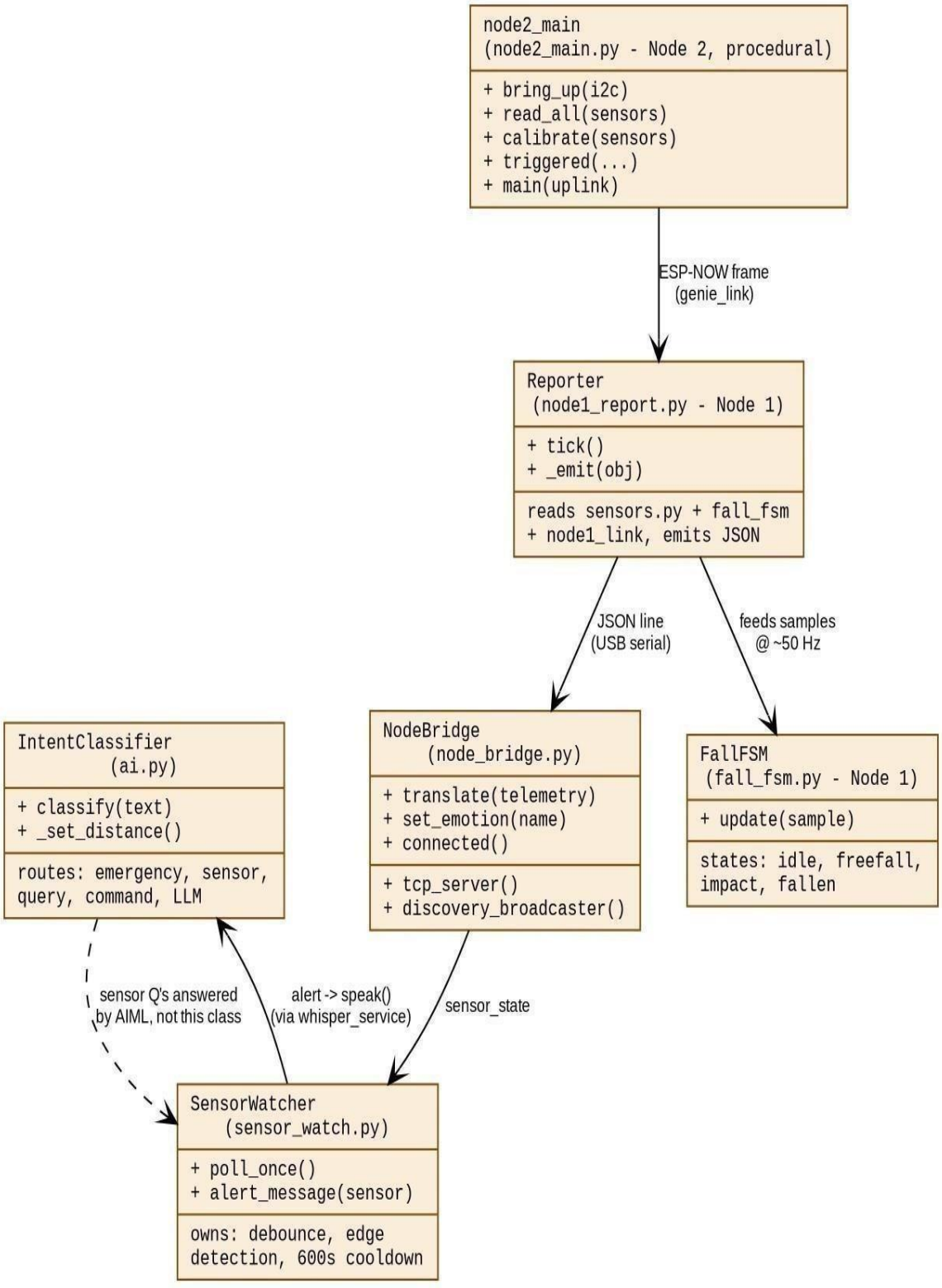


Figure 9: Code Diagram (Class-Level) — GENIE Sensor Pipeline

5.4 ERD

GENIE has no relational database. The diagram below documents its runtime data model instead: the in-memory structures api.py builds from board telemetry and uses to answer sensor queries and drive relay/alert behavior.

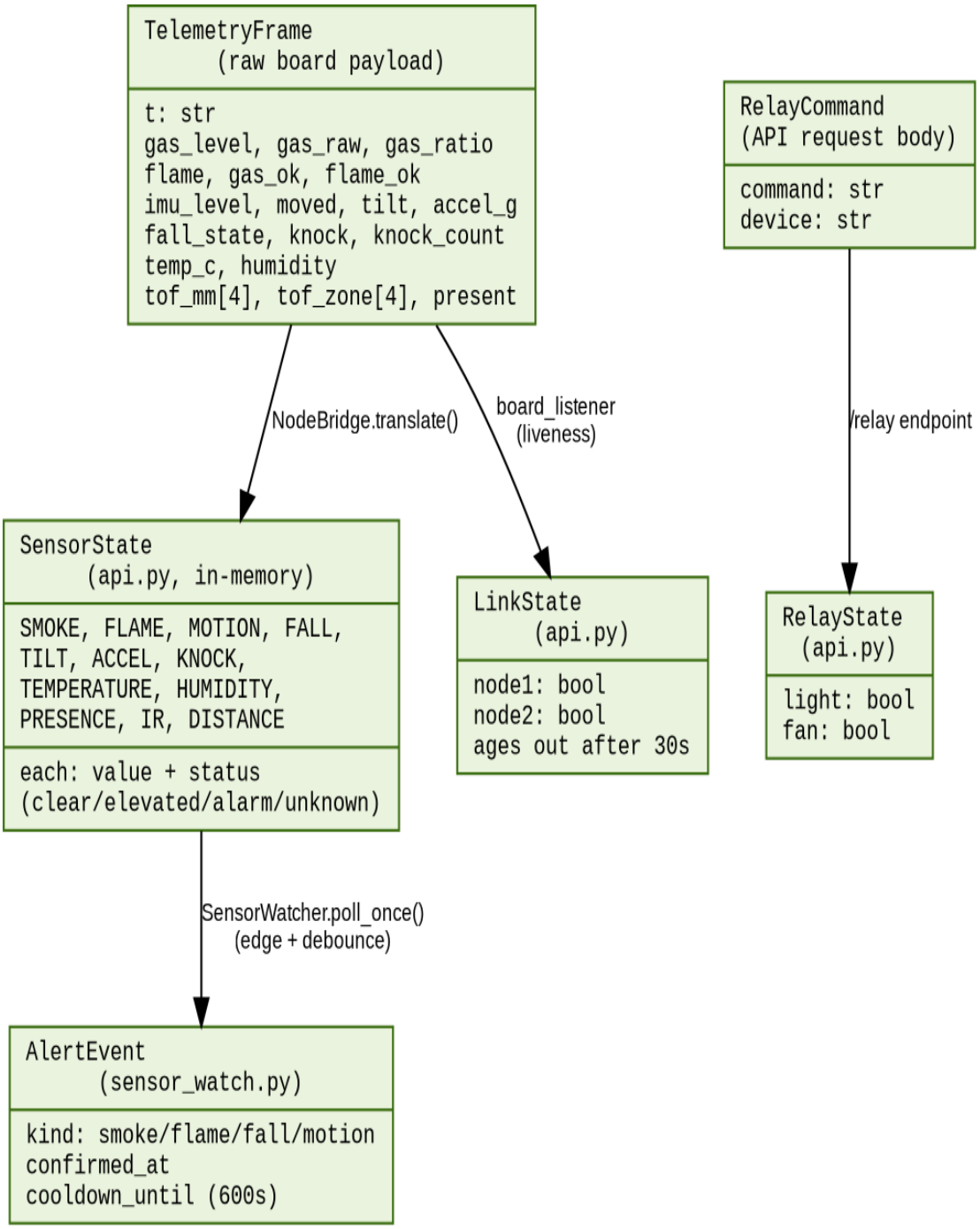


Figure 10: GENIE Runtime Data Model (no relational database is used)

5.5 Data Dictionary

GENIE does not use a relational database; the tables below document the fields that actually flow through the system the raw telemetry frames each board sends, and the sensor-state fields api.py derives from it (see the Runtime Data Model above).

Table 12: Node 1 / Node 2 Telemetry Frame Fields

Field	Source	Meaning	Type / Unit	Notes
gas level	Node 1	Debounced MQ-2 gas/smoke reading	str (clear/elevated/alarm)	Derived from gas raw + gas ratio
gas raw / gas ratio	Node 1	Raw ADC reading and ratio vs. baseline	int / float	Baseline calibration not yet confirmed
flame	Node 1	Flame sensor digital state	bool	-
gas_ok / flame_ok	Node 1	Sensor self-check flags	bool	False means treat reading as "unknown", never "clear"
imu_level / moved / tilt / accel_g	Node 1	Raw dual-MPU-6050 readings	str / bool / float / float	Feeds fall state via Fall FSM
fall state	Node 1	Fall FSM output	str (idle/freefall/impact/fallen)	See Fall FSM code diagram
Knock / knock count	Node 1	Knock/tap detection on the case	bool / int	Not yet wired to a wake trigger
temp_c / humidity	Node 1	DHT11 reading	float / float	-
present	Node 2 (Node	Room presence from the ToF array	bool	Debounced ~6s

tof_mm [4] / tof_zone [4]	Node 2 (Via Node 1)	Per-sensor distance and zone (front/left/right/rear)	int [4] / str [4]	front sensor also drives the legacy "IR " Channel

Table 13: api.py Sensor State Channels (as exposed by /sensor, /sensors)

Field	Source	Meaning	Type / Unit	Notes
SMOKE	gas level	Gas/smoke alarm state	Value + status	status: clear/elevated/alarm/unknown
FLAME	Flame + flame_ok	Fire alarm state	Value + Status	
MOTION	Moved / present	General motion or presence	Value + Status	
FALL	fall state	Fall alarm state	Value + Status	
TILT / ACCEL	tilt / accel_g	Orientation / acceleration	Value + Status	Diagnostic, not alarm-worthy alone
KNOCK	Knock / knock count	Case knock/tap	Value + Status	Diagnostic only
TEMPERATURE / HUMIDITY	temp_c / humidity	Ambient climate	Value + Status	
PRESENCE / IR / DISTANCE	present / tof_zone / tof_mm	Room occupancy detail	Value + Status	

Table 14: On-Device Feature Vector (har_features.py, FEATURE_VERSION 2)

A window consists of 50 consecutive accelerometer readings and a new window is generated every 10 readings, such that consecutive windows overlap by 80%. In practice, the board runs at around 20Hz instead of the 50 Hz requested by the sampler, as the display rendering shares the same cooperative loop. As a result, a window spans about 2.5 seconds of real time. This rate is kept consistent across recordings to preserve the features; however,

this makes the model sensitive to the loop speed at which it is run, and the display's frame rate must not be varied after training.

#	Feature	Meaning	Unit	Used by model
0	acc_mean	Mean acceleration magnitude over the window; about 1.00 at rest whatever the mounting angle	g	No — dropped
1	acc_std	Spread of the magnitude. The single best "was this eventful" number	g	Yes
2	acc_min	Smallest magnitude in the window	g	Yes
3	acc_max	Largest magnitude in the window	g	Yes
4	acc_range	max − min. Separates one big spike from continuous motion when read with acc_std	g	Yes
5	acc_dev1g	Mean |m − 1|. Deviation from gravity, not from the window mean — in free fall the mean is itself wrong	g	Yes
6	acc_energy	Mean (m − 1) Z. Emphasizes large excursions the way acc_dev1g deliberately does not	gZ	Yes
7	acc_jitter	Mean sample-to-sample change. High in a hand (tremor), near zero on a desk at any angle	g	Yes
8	acc_crossings	Normalized mean-crossing count. Shaking crosses often; one shove barely crosses at all	–	Yes
9	acc_max_jerk	The single sharpest sample-to-sample step. A tap is a brief violent edge; a push is softer and longer	g	Yes
10	acc_frac_low	Fraction of samples below 0.70 g — free fall	–	Yes
11	acc_frac_high	Fraction of samples above 1.30 g — impact and vigorous handling	–	Yes
12	tilt_mean	Mean absolute angle from flat	degrees	No — dropped
13	tilt_std	Spread of the angle	degrees	Yes
14	tilt_range	max − min angle	degrees	Yes

15	tilt_delta	Last minus first, signed. Says the unit ended up somewhere new rather than merely having wobbled	degrees	Yes

Table 15: On-device feature vector (16 produced, 14 read by the model)

The extractor still computes acc_mean and tilt_mean, though the model no longer employs them. The two features are both expressed as absolute values: tilt_mean shifted from 85.3° to 78.1° between the two recording sessions (after the board was remounted) a difference of two standard deviations, whereas all other features stayed within 1.2 so that the model had inferred that "idle means about 85 degrees," but session two presented 78. Removing those two features alone raised held-out accuracy from 31.4% to 54.6%. They remain in the vector to keep existing recordings valid, so that no new data collection is necessary.

Table 16: Training Dataset Schema (har_data.csv)

Column	Meaning	Notes
label	The class being performed when the window was recorded	idle, tapped, pushed, turned, picked up, shaken. Stored as performed; merging happens at training time, not at recording time
session	Which recording session the window came from	This column is what makes a session held-out evaluation possible
version	FEATURE_VERSION at the time of recording	The trainer refuses to run on a file mixing versions, because a feature vector's meaning is positional
f0 … f15	The 16 feature values	Written by the same extractor the board runs

6. IMPLEMENTATION DETAILS

6.1 Development Setup

The development environment for GENIE is divided into a Python stack on the laptop and firmware toolchains for the ESP32-S3.

- **Host language and runtime:** Python 3.11 under Windows, running AudioServer.py, api.py, and the Services/ (whisper service, ai, sensor_watch, node_bridge) package.
- **Speech pipeline:** Faster-Whisper for speech-to-text, openWakeWord/ONNX Runtime for the custom-trained "Hey Genie" wake word model, and Piper for text-to-speech.
- **Intent and conversation:** PyAIML for deterministic pattern matching (relay. aiml, sensor. aiml); Ollama for a local Mistral-7B model for open-domain fallback.
- **Web/API layer:** Fast API and Uvicorn (api.py) providing relay control, sensor state, a TCP board listener, and UDP discovery.
- **Firmware for sensor nodes:** Micro Python running on Node 1 (face/sensor board) and Node 2 (ToF board), flashed and iterated over with Thonny or Esp tool.
- **Firmware for audio and relay nodes:** Arduino-ESP32 (C++) driving the Mic, Speaker, Relay/IoT, and GSM Modem nodes.
- **Testing:** An automated suite (86 assertions across node_bridge self-tests, sensor integration, wire-loop, and Wi-Fi uplink tests) that can run without any physical board attached. • Version control: Git/GitHub for both the laptop codebase and sensor firmware.
- **Edge AI training toolchain:** scikit-learn and NumPy on the host are used only for training and scoring the on-device classifier. Neither are required at runtime nor present on the board the trained model is exported as plain Micro Python source, so the boards carry no inference runtime.
- **Data collection and model export:** har_record.py running a guided collection session against a running api.py; har_train.py reading har_data.csv and writing har_model.py directly into the Node 1 upload folder, stamped with the feature version it was trained on.
- **Networking:** A dedicated Wi-Fi access point (SSID: GENIE1) serving the mic/speaker/relay nodes; Node 1 uses direct USB-serial; and ESP-NOW links Node 1 and Node 2.

6.2 Deployment setup

On their GitHub page, developers describe GENIE as a compact local installation as opposed to something hosted on the cloud: the host (laptop) runs the full software pipeline, and the ESP32-S3 boards are flashed once and then left running as dedicated devices around the monitored room. Before any of GENIE's processes run, Ollama and its Mistral-7B model are already installed and running locally.

Issues that came up during deployment, along with how they were addressed:

- **Retraining discipline:** The dataset is re-recorded when either the feature vector or display frame rate change. Jitter, crossings, and jerk are all calculated per-sample, and thus the model depends on the loop rate; a model trained on one rate silently fails, when deployed at another rate, by producing insufficient data. The feature version stamp is a safeguard to prevent this from happening.
- **Duplicated firmware guard:** six copies of the firmware are stored in two or three upload directories, so that each directory contains a full "select all and flash" set. check_pack.py is run before every flash, and it loudly complains if any of the copies differ, printing the commands necessary to repair the directory tree.
- **Wrong PSRAM firmware built for Node 1:** a wrong Micro Python build (without PSRAM support) was flashed to Node 1, causing WIFI related out-of-memory errors on the face board; this was diagnosed via crash logs and later fixed by reflashing the node with the correct build.
- **Intermittent sensor power-supply problem:** Node 2's ToF array sometimes fails to initialize at boot (ENODEV), and the same power rail appears to be involved in the display-panel reset storm on Node 1; there is nothing to do but to wait for the hardware fix (more robust power supply).
- **Dual-microphone synchronization:** streaming the two channels as two separate TCP connections at ports 9000/9003 and 9002/9004 made cross-channel time alignment difficult; we ended up using a single dual-channel TCP stream at port 9000, thus simplifying both capture and the downstream confidence comparison in Whisper.
- **Wake-word false positives on conversation:** the initial confidence threshold was too lenient, allowing wake words to trigger on unrelated conversation; this was calibrated using a small recorded sample set.

6.3 Algorithms

6.3.1 Dual-Microphone Confidence Selection

Faster-Whisper independently transcribes both microphone channels; the channel whose transcription has the higher avg_logprob (i.e., the model is more confident that it captured actual speech rather than ambient noise) is passed on to the intent layer. This allows GENIE to handle the case where one microphone is placed farther away from the speaker or is partially obstructed.

6.3.2 Fall Detection Finite State Machine (Fall FSM)

Node 1 implements a 4-state state machine on the accelerometer data: idle -> freefall -> impact -> fallen. Transitioning out of idle requires a sustained near-zero-g condition indicating a freefall. Additionally, transitioning to fallen requires an additional high-g spike followed by a sustained period of stillness in order to distinguish between a true fall and just dropping the device or a sudden impact.

6.3.3 Sensor Debounce, Edge Detection & Cooldown

Sensor watcher. poll once () is not called on every raw sensor reading. It tracks the previous state of each channel, sending an event only on a transition from clear/alarm. It debounces every sensor individually (4 s window for smoke, 6 s for motion/presence) before registering an alarm, and then applies a 600 s cooldown between alerts of the same type to avoid both false positives and spamming when a hazard has already been taken care of.

6.3.4 Gas Baseline Ratio

Rather than comparing some ADC value on the MQ-2 to a particular value, GENIE calculates the gas ratio in comparison to a rolling baseline and then interprets the resulting percentage as clear/elevated/alarm. (Note that this baseline is not calibrated against a known source of gas/smoke see Limitations.

6.3.5 Deterministic-First Intent Routing

Intent classifier. classify () first checks a transcribed command against the compiled AIML pattern files; only an unmatched, non-sensor query is passed along to the local LLM for processing. Sensor-related queries are not passed on at all, specifically to avoid the possibility of the LLM generating a response based on a sensor reading that the system has not actually observed.

6.3.6 Wake-Word Run-Length Gating

A raw wake-word score alone is not sufficient to trigger a recording. It has to exceed the threshold for a number of consecutive analysis blocks, after which a single block with a score below the threshold resets the count to zero. The reason for this is that a wake word uttered by a speaker will last for approximately half a second, and therefore it will cover multiple blocks with a consistent score above the threshold. On the contrary, a false positive such as a knock, a crash, or a word with one syllable of unrelated utterance will peak at one block and then drop in the next, thus resetting the counter. In this way, the run-length encoding procedure prevents false positives by eliminating short-term spikes, which would have occurred if the threshold was simply set higher. As explained in the lecture, increasing the threshold would result in a significant reduction, but not the complete removal of false positives, and such an approach would not prevent waking up by a word.

6.3.7 Voice Activity Gating with an RMS Floor

A frame is considered to contain speech if the voice-activity detector agrees and the level within the frame, estimated over a short window, is above minimum. Two ratios of speech frames to background frames determine the start and end of capture; the former is higher than the latter, so a command is easier to start than to stop, allowing distant conversation to keep the microphone open. The level floor is adjusted according to the channel used for detection, because averaging two channels reduces the measured level by approximately 20%, and a floor set on one channel would act as a stricter gate on the other.

6.3.8 Follow-Up Conversation Window with a Chain Cap

After the system stops speaking, a short window in which sustained speech alone would trigger a new capture commences, such that a multi-turn conversation incurs only one wake phrase cost, instead of one per turn; thus, reducing the overall cost and inconvenience. The window is determined from each audio chunk's current time, comparing it against the timestamp when speech output ended, rather than being latched on a state transition; thus, ensuring that a lost signal doesn't result in a microphone being permanently opened. Two additional safeguards apply. A brief arming delay after speech ended covers room reverberation and playback buffer drain. A hard cap on consecutive wake-word-free captures then provides a structural guarantee against acoustic feedback: a loop requires an unbounded chain of such captures, and after the cap the wake phrase is again required. This difference matters, as timing margins are typically probabilistic and a feedback loop needs them only to be wrong on one occasion, whereas the cap terminates the chain unconditionally.

6.3.9 Windowed Feature Extraction for On-Device Recognition

A feature Window keeps a fixed ring buffer 50 magnitude samples, 50 tilt samples and returns a 16-value summary every 10 samples, overlapping by 80%. A shove lasts between roughly 60ms and 800ms and with no overlap can straddle a boundary, appearing as half shove in two windows; at this overlap is whole in at least one. A missing sample discards the window instead of shortening it, jitter, jerk, crossings and tilt_delta are all adjacency measures which are meaningless across a gap. The statistics are recomputed fresh over the ordered buffer, not incrementally, since a subtly wrong running standard dev is exactly the bug this design is meant to avoid.

6.3.10 On-Device MLP Inference and Confidence Gating

The exported model is a 14-12-4 multi-layer perceptron 232 parameters written as plain Python lists, together with per-feature mean/scale from the training set/frozen. predict() takes the whole 16-value vector, picks its own 14 columns, never ask the caller which were dropped, standardises with the frozen constants, applies one ReLU hidden layer, softmax for output and returns a class name with a confidence. Recalculating standardisation from live data would have been more wrong than just a different value: tilt_std ranges from 0.28 to 16.3 across the classes, acc_std 0.0037 to 0.39, so un-scaled tilt features overwhelm everything else. Below a confidence of 0.70 the board outputs nothing at all, following the

same principle that makes an unreadable sensor report "unknown", not "clear": silence is a legitimate answer, but wrong reaction is not.

6.3.11 Session-Held-Out Evaluation, Feature Dropping and Class Merging

Accuracy on a random split of a single recording session is not a valid metric here. Consecutive windows share 80% of their samples, so a test set's window can share 40 of its 50 with a training set's window and it's more of a memory test than a generalization test. The trainer then, reports both splits and states the session-held-out one is the one to cite. Two decisions come directly from that number rather than preference. The two absolute valued features are removed, because they shifted when the board was remounted between sessions. The three classes the sensor set can't distinguish tapped/pushed/turned are combined into a "contact" class during training. Rotating the board flat on a desk doesn't change its angle from flat, so the one-feature meant to detect reorientation can't, what remains is hand motion while touching, precisely a push. Separating tapped and pushed depends on max_jerk (per sample), the loop ran about 8% slower in the second session, so jerk itself changed. Merging is done in the trainer, not the recorder, so the six performed labels stay in the CSV, a different merge or none, once a surface-contact sensor is wired in, requires a re-run instead of a re-record.

6.3.12 Handling-Recognition Finite State Machine (Rule-Based Baseline)

handling_fsm.py generates the same five events from hand-tuned thresholds. It uses jitter instead of absolute tilt (a specific reason), the face board is mounted at an angle and reads about 104° at rest, so any absolute angle rule needs a calibrated idea of "normal" and fails once the unit is remounted or facing a different direction. Jitter needs no baseline; it is high in a hand and near zero on any surface at any angle; "set down somewhere new" resolves on its own. The FSM also encodes three suppressions the classifier inherits: an active fall overrides everything, a knock suppresses pushes (short window) because tapping the case spiked the magnitude to ~1.5g, a warmup period after boot stays silent (unit is genuinely being handled while someone powers it up).

6.3.13 Delay-and-Sum Microphone Beamforming

Both mics are on one shared I2S bus in stereo mode, so the two channels are already sample synchronous; no resampling or clock alignment is needed, which is what makes simple averaging valid at all. The summed channel is what the wake word and the VAD listen to. The speech in both channels is correlated, adds coherently, each mic's own electrical/thermal noise is uncorrelated, partly cancels, providing up to ~3dB of SNR gain. Without delay compensation, the array points broadside, a speaker roughly in front of the unit is combined exactly. At 10cm spacing, the max path difference is 292us, 4.7 samples at 16kHz. The first cancellation null is at 6.6kHz at 15° off axis, 3.4kHz at 30°, 1.7kHz at 90°. The result is acknowledged, not hidden: the array helps in front, worsens from the side; the proper remedy if off-axis capture is important, is delay estimation followed by align and-sum, not a higher wake-word threshold. Both raw channels are saved to separate

WAVs, Whisper transcribes each one independently, the one with higher confidence is forwarded.

6.3.14 Speech Output Wave Shaping

Synthesized speech arrives with a big gap between its peak and average levels: a few consonant transients peaked right at the ceiling, but the bulk of the speech was well under, so increasing the gain can only cause distortion. The output is sent through a memoryless wave shaping curve which narrows the gap, not raises the ceiling. Memoryless curve was deliberately selected, since it maps each sample irrespective its neighbors, so processing the audio in streaming chunks is identical to processing the full utterance, makes no discontinuity at chunk boundaries (a compressor with attack/release would need lookahead that the streaming path can't supply). Limiting is done before any destination-specific attenuation, so the curve always works on the full-scale signal.

6.3.15 Single-Decision Climate Control with Hysteresis

The obvious approach: one rule per condition, hot switches the fan on, humid switches the dehumidifier on, but fails at first two rules target the same relay, nothing determines which one wins. With a single physical fan, "hot" and "humid" are two reasons for the same switch. decide () thus takes all readings at once, returns exactly one desired state and the reason for it, so no conflicts possible. Each condition has two thresholds, act at 32°C, release at 26.5°C; act at 70% humidity, release at 67%, so a room hovering on a boundary doesn't toggle the relay every poll. A milder band (28°C) gives a spoken suggestion, leaves decision to the person. A missing reading returns no opinion rather than False, so a DHT11 that stops responding holds the previous decision, not appear as a cold/dry room.

6.3.16 Event-Counter Draining for Handling Reactions

sensor_watch reads state, asks whether it is bad; reaction watch reads moments, asks something just occurred. A state can be re-read a second later and still hold true, but a shove can't, which is why api.py keeps an append-only event log with a steadily advancing cursor, not a flag. A flag is either missed between polls or spoken twice. Three rules: On its first poll the cursor jumps to the newest entry, instead of replaying from zero, so GENIE never starts with a complaint about a shove that happened before the laptop was running. If several events land in one window: only the latest is spoken, because "lifted → still lifted → set down" means the unit is already back on the table. If a reaction arrives while GENIE is speaking: it's dropped, not queued, because a comment about being nudged is useless thirty seconds late.

6.3.17 Emergency Escalation with Acknowledgement

An alert first sends an SMS and then opens a 45s window for ack before dialing, since the text explains what occurred, whereas a ring only signals that something did. Once activated, the modem node handles the entire escalation independently, only the spoken in room announcement requires the host, and the trigger has already arrived via that link, so little

remains at risk by the time it becomes critical. The contact may press 1 to acknowledge and halt the escalation, or press 2 to forward it to the next contact. One strict rule governs the implementation: the ack window and the inbound-SMS command relay must never run concurrently, since both access the modem's unread list, reading it marks those messages as read whichever queries first takes the messages the other needed, and a legitimate ack could be handled by the LLM instead of stopping the call.

6.4 Constraints

6.4.1 Assumptions

- All nodes (mic, speaker, relay) connected to Wi-Fi are on the same local network (GENIE1) as the host laptop; Node 1 is connected to the host over USB-serial.
- Ollama is installed and running locally on the host machine and has the Mistral-7B model downloaded and ready to go before any of GENIE's processes are run.
- All nodes are powered by the mains (no batteries yet).
- Only one person is talking at a time, with minimal background conversation near the microphones.

6.4.2 System constraints

- The whole voice and safety pipeline has to function independent of an Internet connection.
- The host pipeline needs Python 3.11 on Windows.
- Each ESP32-S3 board is constrained by their own GPIO/I2C budget and this is the reason why the design was distributed as a multi-node architecture rather than a monolithic board.
- An autonomous emergency calling requires a SIM registered with the local carrier (PTA/DIRBS in Pakistan) in the GSM modem.

6.4.3 Restrictions

Being the academic FYP, not the client commissioned system, GENIE's constraints are dictated by the FYP-2's evaluation criteria, rather than an external client: the project needs to adhere to the documentation requirement specified by the university's FYP-2 template, and have two particular capabilities as evaluation gates: a sign-in web dashboard and battery powered operation.

6.4.4 Limitations

- No web dashboard; the only way to review system/sensor status is via the API or the console, not a browser UI.
- No battery-backed operation; all nodes require mains power.
- The MQ-2 gas/smoke baseline has not been calibrated against a real smoke or gas source; smoke level readings are not yet validated as safe/reliable in an actual fire.

- Fall detection has been validated in unit tests but is sample-starved on real hardware so far (fallrate: false in current logs); it has not yet been confirmed to catch a real fall reliably.
- Node 2's ToF array fails intermittently at boot due to the open power-supply issue, so presence detection is not yet reliable end-to-end on hardware.
- Autonomously initiated GSM emergency calling has implemented firmware but has not been validated end-to-end, pending SIM network registration.
- No knock/hand-wave wake trigger the hardware exists (knock/knock count) but is not wired to any action.
- No camera-based (ESP32-CAM) visual monitoring.
- No multi-user voice differentiation: GENIE cannot tell which person is speaking.
- The on-device classifier is trained using one unit, one mounting position, and a limited set of operators over two sessions; the 72.6% held-out figure reflects what this build achieves on a second day, not a claim about a different unit or person.
- Three of the six handling classes tapped, pushed, and turned are combined because the current sensor set cannot tell them apart: distinguishing them requires a surface contact signal like the SW-420, which detects the desk and goes quiet when the unit is lifted; that sensor is installed but is not yet part of the feature vector.
- The feature vector depends on the board's loop rate; jitter, crossings, and jerk are per sample measures, so changing the display frame rate or adding work to the cooperative loop alters them, and the dataset would have to be recorded again.
- The dataset is imbalanced after merging (766 contact windows vs. 244 shaken), so overall accuracy flatters the model relative to its perclass recall; per-class figures are reported in Chapter 8, for that reason.
- sensor_watch initializes its cooldown clock to zero and compares it against a monotonic clock; on a host that has been running for less than the 600-second cooldown, the very first alert of a session can be suppressed; this does not happen on a normally running laptop and is a one-line fix, but it is a real defect and is documented here rather than left to be discovered later.
- Autonomous climate control acts only on the fan relay: nothing in the face board suggests whether a light should be on (that becomes possible once the ToF presence array is reliable) because someone being in the room justifies a light in a way that temperature never does.

7. PROJECT MONITORING, CONTROL AND TRACEABILITY

7.1 Traceability Matrix

7.1.1 Legend Prototype / Component IDs (PID)

- P1 Microphone / Speaker Node firmware
- P2 Relay/IoT Node firmware
- P3 Node 1 (Face/Sensor Board) firmware
- P4 Node 2 (ToF/Presence Board) firmware
- P5 AudioServer.py + speech pipeline (wake word, Whisper, Piper)
- P6 AIML engine + Ollama/Mistral-7B intent routing (ai.py)
- P7 api.py (relay/sensor API, telemetry bridge)
- P8 Modem Node firmware (GSM emergency calling)
- P9 Edge AI stack (har_features.py, har_model.py, har_train.py, har_record.py) shared feature extractor, on-board model, trainer, collector
- P10 Autonomous watchers (sensor_watch.py, reaction_watch.py, climate_watch.py) the three system-initiated behaviors
- P11Verification tooling (fake_node.py, sensor_monitor.py, serial_bridge.py, check_pack.py)

7.1.2 Legend Test Artifact IDs (TID)

- T1 node_bridge.py self-tests (automated)
- T2 Sensor integration test suite (automated)
- T3 Wire-loop end-to-end test (automated)
- T4 Wi-Fi uplink test (automated)
- T5 Handling-chain test suite (automated, 18 assertions)
- T6 Session-held-out model evaluation report (har_train.py, automated)
- T7 AIML structural validation (XML well-formedness, duplicate-pattern scan, default branch scan)
- T8 Manual bench and voice QA (human-in-the-loop)

7.1.3 Requirements vs Prototype (PB-ID vs PID)

Complete the acceptance criteria are met and demonstrated;

- Partial the code is written and passes its automated tests, but a stated external dependency blocks final demonstration (each such row names its blocker);
- Not started declared out of scope for this FYP-2 iteration and carried forward.

PB-ID	Requirement (short form)	PID	Implemented by	Status
EP101	Zero-configuration UDP node discovery	P1, P2	UDP broadcast responder in api.py and AudioServer.py; node-side discovery in each board's firmware	Complete

PBID	Requirement (short form)	PID	Implemented by	Status
EP10 2	Distributed mic and speaker hardware nodes	P1	Mic node (dual INMP441, shared I2S bus, stereo stream); speaker node (MAX98357A) driven by the opcode protocol	Complete
EP201	Dual-microphone capture and confidence routing	P5	AudioServer.beamform() for detection; whisper service. transcribe_pair_and_print () compares avg_logprob per channel	Complete
EP20 2	Wake-word gated listening	P5	Services/wake_word.py openWakeWord/ONNX, custom hey_genie. onnx, threshold 0.65 sustained over 3 consecutive blocks	Complete
EP20 3	Hybrid deterministic-first intent routing	P6	Services/ai.py Intent Classifier over relay. aiml and sensor. aiml, with Mistral-7B fallback; sensor questions never reach the LLM	Complete
EP30 1	Voice-controlled appliance switching	P2, P7	relay. aiml patterns → api.py /relay → active-LOW relay node	Complete
EP30 2	Multi-sensor hazard monitoring and telemetry	P3, P4, P7, P10	node1_report.py emitter, fall_fsm.py, node_bridge. Translate(), api.py sensor state, sensor_watch.poll_once()	Partial blocked by sensor-board power supply (ToF ENODEV)
EP30 3	Autonomous emergency call and SMS	P8, P10	sensor_watch → TCP trigger → esp32/modem_main.py; SMS, acknowledgement window, then voice call	Partial blocked by SIM registration (PTA/DIRBS)

EP40 1	Expressive animated face	P3	eyes.py and genie_ui.py on the ILI9341; emotion channel from the host, with alarms overriding any requested mood	Partial emotion channel unconfirmed on hardware
EP501	Sign-in web dashboard	—	No component carried forward	Not started
EP502	Battery-powered node operation	—	No component carried forward	Not started

EP601	On-device handling classifier	P9, P3	Har features. feature Window and Har model. predict () running in Micro Python on the face board, fed by node1_report. poll fall ()	Complete
EP602	Session-held-out model evaluation	P9	har_train.py reports random-split and held out scores side by side; har_record.py stores a session id on every row	Complete
EP603	Confidence gating (abstention)	P9	CONFIDENCE_MIN = 0.70 in har_model.py; no event is emitted below it	Complete
EP604	Learned vs. rule-based comparison	P9, P3	handling_fsm.py as the tuned baseline; har_train.py additionally trains and scores a decision tree	Complete
EP701	Autonomous climate control	P10, P7	climate watch. decide () one decision over all readings, two thresholds per condition, spoken reason, user override	Complete
EP702	Spoken reaction to being handled	P10, P6, P7	api.py event log with cursor → reaction watch → ai. build_reaction_reply () → sensor. aiml random sets	Complete
EP703	Follow-up window without a second wake word	P5	AudioServer.followup_listener () 8 s window, chain capped at 3, silence on unclear audio inside the window	Complete
EP704	Two-way SMS relay and DTMF acknowledgement	P8, P6	modem_main.poll_inbound_sms() and the host's handle_incoming_sms(); DTMF 1 acknowledges, 2 escalates	Partial blocked by SIM registration (PTA/ DIRBS)

NFR0 1	Fully offline operation	P5, P6	Faster-Whisper,AIML, Ollama/Mistral-7B and Piper all run on the host; no telemetry or audio leaves the local network	Complete
NFR0 2	Sub-second fall and flame alerting	P3, P7	Confirmation happens in Node 1 firmware before the line is emitted; debounce override is 0 s for FLAME and FALL	Partial measured through the loop test;
NFR 0 3	Debounced smoke and presence alerting	P10	SENSOR_ALERT_DEBOUNCE 4s with per-sensor overrides; presence debounced across thee ToF array	Complete
NFR 0 4	A silent sensor never reads as safe	P3, P7, P6	gas_ok / flame_ok checked before the reading; "unknown" belongs to no alerting set and lands on sensor.aiml default branch	Complete
NFR 0 5	Automated	P11	Five automated suites plus fake_node.py's twelve board	Complete
	regression coverage without hardware		scenarios, all runnable with no board attached	
NFR 0 6	Zero manual network configuration	P1, P2	UDP discovery on port 9999; no hardcoded addresses in firmware	Complete
NFR 0 7	Battery-backed operation	—	No component — carried forward	Not started
NFR 0 8	Inference inside the board's loop budget	P9	232 stored parameters, pure Micro Python, fixed ~800-byte ring buffer, no inference runtime and no dynamic allocation per window	Complete
NFR 0 9	Model / extractor version guard	P9	FEATURE_VERSION compared at import in har_model.py; one shared extractor file imported by both board and trainer	Complete
NFR 1 0	Autonomous actions are explained and overridable	P10	climate watch emits a spoken reason with every relay change; a spoken override holds until the situation band changes	Complete

Table 17: Requirements traced to implementing components (PB-ID vs PID)

7.1.4 Requirements vs Test Cases (PB-ID vs TID)

PB-ID	Verified by	Type	Evidence / observed result	Verdict
EP101	T4, T8	Auto	WiFi uplink suite passes 21/21; nodes acquire the host address on boot with no static configuration	Pass
EP102	T8	Manual bench	1,941 captured recordings in recordings/processed, dualchannel throughout	Pass
EP201	T8	Manual bench	Both channels transcribed per capture; the winning channel and both avg_logprob values are logged for every utterance	Pass
EP202	T8	Manual bench	Custom model trained from 242 clips across 5 speakers; gate is 0.65 sustained over 3 consecutive 80 ms blocks, raised from 2 after false accepts persisted	Pass
EP203	T7, T8	Auto	sensor.aiml: 335 categories, 335 patterns, 0 duplicates, valid XML. relay.aiml: 27 categories, 0 duplicates. Sensor intents never reach the LLM	Pass
EP301	T8	Manual bench	Spoken command to relay actuation confirmed on hardware	Pass

PB-ID	Verified by	Type	Evidence / observed result	Verdict
EP302	T1, T2, T3, T8	Auto + manual	86 assertions across the bridge, integration and wire-loop suites pass; the wire loop drives the real firmware emitter into the real reader and the real API. Hardware bring-up outstanding	Pass (software) / blocked (hardware)
EP303	T8	Manual bench	SMS and call sequence exercised against the modem; radio pinned to 2G after LTE→GSM handover dropped every call. Live validation needs a registered SIM	Blocked (external)
EP401	T8	Manual bench	Display renders at 2,477 kpx/s and sustains 118 fps in soak; the host-set emotion channel is plumbed but not yet confirmed on hardware	Partial
EP501	—	—	Not implemented in this iteration	Not started
EP502	—	—	Not implemented in this iteration	Not started

EP601	T5, T6	Automated	Handling-chain suite passes 18/18; 1,532 labelled windows produced by the same extractor the board runs	Pass
EP602	T6	Automated	Random split 88.3% (MLP) and 84.1% (decision tree) on 383 windows; sessionheld-out 72.6% on 679 windows never seen. The 15.7-point gap is reported, not hidden	Pass
EP603	T6	Automated	At the 0.70 gate the model answers 77.8% of held-out windows and is 82.6% correct on those, against 72.6% if it answers every window	Pass
EP604	T6	Automated	Decision tree, MLP and the hand-tuned FSM all reported; the MLP beats the tree by 4.2 points on the random split	Pass
EP701	T8	Manual bench	Fan switches at the act threshold and releases at the lower one; a spoken override holds across polls; a null reading holds the previous decision	Pass
EP702	T5	Automated	All five FSM event names have a sensor.aiml line and are accepted by api.py; an unknown event name yields silence rather than an empty utterance	Pass
EP703	T8	Manual bench	Follow-up window opens after speech ends, caps at 3 chained turns, and stays silent on unclear audio so the window can close	Pass
EP704	T8	Manual bench	Relay and reply path exercised; inbox polling proven to be suspended during an alert. Live validation needs a registered SIM	Blocked (external)
NFR0 1	T8	Manual bench	Full voice and safety loop exercised with the internet disconnected	Pass
NFR0 2	T2, T3	Automated	Flame and fall confirmed to bypass the MQ-2 debounce (override 0 s); measured chain latency 50–150 ms for fall and flame, ~4 s for smoke by design	Pass (bench)
NFR0 3	T2	Automated	Smoke holds the 4 s settle; a latched alarm does not re-fire per poll; a new episode fires again after it clears	Pass

NFR0 4	T2, T7	Automated	A faulty channel reports "unknown"; all 24 conditions in sensor.aiml carry a bare default branch, so no unreadable sensor can produce an all-clear	Pass
NFR0 5	T1–T5	Automated	104 assertions across five suites, all passing, with no board attached; fake_node.py adds 12 scriptable board scenarios	Pass
NFR0 6	T4	Automated	WiFi uplink suite passes 21/21 against a real socket	Pass
NFR0 7	—	—	Not implemented in this iteration	Not started
PB-ID	**Verified by**	**Type**	**Evidence / observed result**	**Verdict**
NFR0 8	T6, T8	Auto + manual	232 parameters; feature vectors observed arriving at a stable rate (17, 16, 16, 16, 16, 16 across six consecutive takes) with the display loop running	Pass
NFR0 9	T5, T6	Automated	A version mismatch raises at import; the trainer refuses a CSV mixing feature versions	Pass
NFR1 0	T8	Manual bench	Every autonomous relay change is accompanied by a spoken reason naming the reading	Pass

Table 18: Requirements traced to verification (PB-ID vs TID)

Measure	**Value**
Backlog items in this matrix	29 (19 functional, 10 nonfunctional)
Items with a named implementing component	26 of 29 (89.7%)
Items with at least one automated test artefact	17 of 29 (58.6%)
Items verified end to end	21 of 29 (72.4%)
Items implemented but externally blocked	5 of 29 (17.2%)
Items declared out of scope for this iteration	3 of 29 (10.3%)

8. RESULTS/OUTPUT/STATISTICS

8.1 Completion how much of the backlog is built

The backlog contains 29 items: 19 functional stories across seven epics, and 10 nonfunctional requirements. Each was assigned one of three states in Chapter 7.

State	Meaning	Count	%
Complete	Acceptance criteria met and demonstrated	21	72.4%
Partial	Code written and passing its automated tests; a named external dependency blocks final demonstration	5	17.2%
Not started	Declared out of scope for this iteration and carried forward	3	10.3%

Table 19: Backlog completion by state

Measure	Calculation	Result
Verified completion	21 complete ÷ 29 total	72.4%
Weighted completion (partial counted at 50%)	(21 + 2.5) ÷ 29	81.0%
Completion excluding the three declared gates	21 ÷ 26 in scope items	80.8%
In-scope items with a working implementation	26 ÷ 26 in scope items	100%

Table 20: Derived completion figures

The three items marked as not started the sign-in web dashboard (EP5-01) and batterypowered operation (EP5-02, NFR-07) were identified as remaining evaluation gates from the beginning, not found late, and are covered in Chapter 10.

Each of the five partial items is held back by a factor external to the software, and it is useful to clarify exactly what that is:

PB-ID	What is complete	What blocks demonstration
EP3-02	Full telemetry chain, translation and alert policy; 86 assertions passing	An intermittent power-supply fault causes the ToF array to fail bringup non-deterministically (ENODEV on a different random subset each boot)
EP3-03	Modem firmware, escalation sequence, acknowledgement window	The SIM is not yet registered with the local carrier (PTA/DIRBS)
EP4-01	Display driver, expression set, host-to-board emotion channel	The emotion channel has not been observed on real hardware
EP7-04	SMS relay in both directions, DTMF handling, inboxpoll exclusion during alerts	Same SIM registration dependency as EP3-03
NFR-02	Confirmation in firmware, zerodebounce override for flame and fall, latency measured through the loop test	End-to-end timing on assembled hardware awaits the power fix

Table 21: Partial items and their blockers

The difference is critical to how this project should be interpreted. None of the requirements within scope are unimplemented; the remaining issues are a power-supply fault, a SIM registration, and one unobserved display channel. The power fault diagnosis is based on evidence and not speculation: a different random subset of four independent sensors failing on each boot can't be a code defect, which would fail identically every time, nor any wiring fault, which would always fail on the same sensor. Only a supply sagging near its threshold gives a different response each run and the same rail is implicated in a display reset storm on the face board that rose from 18 occurrences in one run to 86 in the next.

8.2 Accuracy how well the AI components perform

GENIE includes four inference phases: wake-word detection, speech recognition, intent routing, and the on-device handling classifier. The final phase is the one with a formal, reproducible accuracy measurement, and it is reported first and in full.

8.2.1 On-Device Handling Classifier

Dataset. Two recording sessions were made on two different days, the board being remounted between recordings, giving altogether 1,532 labeled feature windows. Seventeen calibration rows were removed from the data, resulting in the set described in Table 8.3. At training time, three out of six performed classes were merged to one "contact" class, as described in Chapter 6.

Class (after merge)	Windows	Share	Performed classes merged in
contact	766	50.0%	tapped, pushed, turned
idle	275	17.9%	—
picked_up	247	16.1%	—
shaken	244	15.9%	—
Total	1,532	100%	Session 1 (28-08): 853 windows · Session 2 (01-09): 679 windows

Table 22: On-device dataset composition

After merging, the data is imbalanced at 766 to 244, which favors overall accuracy, and makes individual accuracy per class under-represented; thus, per-class are reported instead of an aggregate.

Two models were trained and both are presented because they fail in different ways. A decision tree outputs nested conditionals, does not require feature scaling, and can be printed to reveal the exact rule that triggered. An MLP is often more accurate and handles unseen data more gracefully, but it is an opaque set of numbers. Each was evaluated twice.

Evaluation	Model	Test windows	Accuracy	Interpretation
Random split	Decision tree (depth 6)	383	84.1%	Optimistic do not quote
Random split	MLP 14-12-4	383	88.3%	Optimistic do not quote
Session held out	MLP 14-12-4	679	72.6%	The real number trained on session 1 only, tested on a session never seen

Table 23: Handling classifier accuracy

The 15.7-point difference between the two assessments is a visible product of the overlap effect. Consecutive windows are 80% overlapping in their samples, meaning a window in a random test split could share 40 of its 50 samples with a training window, of which the model has already seen, so the score is more like a memory test. If only the random split was reported the headline accuracy of the project would be overstated by more than fifteen points, the held-out figure represents the true performance of the device.

The process used to derive the held-out figure. The first ever measurement of the six classes as recorded, was 31.4%, just above the 16.7% chance threshold. Every subsequent improvement was the result of a diagnosis, not the tuning of parameters.

Change	Reason	Held-out accuracy
Drop tilt_mean	It moved 85.3° → 78.1° when the board was remounted (2.0 standard deviations, while every other feature stayed inside 1.2), so 107 of 108 idle windows were misclassified	53.2%
Drop acc_mean	Absolute-valued for the same reason; every remaining feature is a spread, a difference or a rate and so means the same thing at any mounting angle	54.6%
Merge tapped, pushed, turned	Rotating the board flat on a desk does not change its angle from flat, so no feature can see it; tapped depends on max_jerk, a per-sample measure that shifted when the loop ran ~8% slower in session 2	72.6%

Table 24: Held-out accuracy by diagnosis

Actual ↓ / Predicted →	contact	idle	picked_up	shaken	Recall
contact	302	7	29	11	86.5%
idle	16	87	5	0	80.6%
picked up	72	10	26	0	24.1%
shaken	36	0	0	78	68.4%

Table 25: Confusion matrix, held-out session

The clear weak point is picked up at 24.1% recall due to a missing sensor rather than a missing layer. An accelerometer is unable to distinguish between "in a hand, held fairly still" and "on a desk, being touched"; surface-contact sensors are able to do so because the SW-420 hears the desk and goes quiet the moment the unit is raised off it. Both SW-420 modules are mounted on the board but are not yet included in the feature vector adding them is the single most valuable change available to this model, and is stated directly rather than being hidden behind an aggregate metric.

Confidence gating. Rather than responding to every window, the board does not speak when the SoftMax confidence drops below a threshold. The trade-off was measured, not guessed.

Confidence gate	Windows answered	Accuracy when it answers	Windows left silent
None (answer everything)	100%	72.6%	0%
0.60	85.3%	79.6%	14.7%
0.70 (shipped)	77.8%	82.6%	22.2%
0.80	69.2%	85.5%	30.8%

Table 26: Confidence gate trade-off, held-out session

0.70 was selected as it gives 10 accuracy points for 1/5th of the windows, and a device that doesn't talk is safe, while one that speaks when it shouldn't is not. We use the same principle throughout the safety layer, as a sensor that can't be read would report unknown, rather than clear.

Cost on the device.

Table 27: On-device model cost

Property	Value
Architecture	14 inputs → 12 hidden (ReLU) → 4 outputs (softmax)
Parameters	232, stored as plain Python lists
Inference runtime	None pure MicroPython; no TensorFlow Lite, no numpy, no custom firmware build
Working memory	One fixed ring buffer of 50 magnitudes and 50 tilts, about 800 bytes; no per-window allocation
Window	50 samples, new vector every 10 samples (80% overlap)
Observed vector rate	Stable across takes 17, 16, 16, 16, 16, 16 vectors over six consecutive recordings, with the display loop running
Safety guard	FEATURE_VERSION checked at import; a mismatched model refuses to load rather than reading the wrong column

8.2.2 Speech Front End and Intent Layer

The remaining three inference stages are presented according to configuration, dataset, and structural coverage. A formal evaluation of word-error-rate and false-accept-rate on a held-out audio corpus has not been conducted, and no such number is asserted here.

Stage	What was measured	Result
Wake word	Training data and gate configuration	Custom "Hey Genie" model trained from 242 clips across 5 speakers; fires only on 3 consecutive 80 MS blocks scoring above 0.65. The consecutive block requirement was raised from 2 to 3 after false accepts persisted at 2
Voice activity	Gate design and tuning	webrtcvad at aggressiveness 3, combined with a per-frame RMS floor of 275. Neither alone is sufficient: webrtcvad misclassifies quiet room noise as speech, and an RMS floor cannot separate a loud bang from a loud word
Speech recognition	Rejection behavior	Faster-Whisper base/int8; segments above a nonspeech probability of 0.6 or below an average log-probability of −1.0 are discarded rather than passed on as Text
Intent routing	Structural coverage (automated scan)	sensor. aiml: 335 categories, 335 unique patterns, 0 duplicates, valid XML. relay. aiml: 27 categories, 0 duplicates. All 24 conditional blocks in sensor. aiml carry a bare default branch
Corpus captured	Volume of real audio processed	1,941 recordings in recordings/processed, captured through the full wake-word and VAD chain

Table 28: Speech front end and intent layer

The 24-of-24 default-branch figure is more interesting because it makes an accidental all-clear impossible rather than merely unlikely: every conditional in the spoken-answer file has a default branch, so an unreadable sensor returns “I cannot read that right now” and never falls through to silent and “everything is fine”.

8.3 Correctness does it do what the requirements say

Correctness is verified by five automated suites that run with no hardware attached, plus a set of structural invariants checked directly against the source

Suite	What it proves	Assertions	Result
node_bridge.py self-tests	The board-to-API field translation, in isolation	included below	Pass
test_sensor_integration.py	Alert policy end to end: edge detection, per sensor debounce, cooldown, unknown versus-clear	51	51 pass
test_wire_loop.py	The real firmware emitter → a fake serial port → the real reader → the real bridge → the real API	14	14 pass
test_wifi_uplink.py	The WiFi telemetry path against a real socket	21	21 pass
test_handling_chain.py	Every handling event the board can emit has a spoken line and is accepted by the API; unknown events yield silence	18	18 pass
Total	—	104	104 pass (100%)

Table 29: Automated test results

The wire-loop suite is the one that deserves special mention. Each of the other tests checks one side against a hand-coded example of what the other is supposed to send, and that is exactly what caused the earlier project's ESP-NOW link to fail: one side sent 10 bytes and the other was looking for 12, and both sets of code worked fine on their own, and all frames were lost silently without either side registering an error. The wire-loop test never writes a protocol line by hand: the bytes are generated by the emitter that will actually run on the board, and received by the reader that will actually run on the laptop

Invariant	Why it matters	Verified
Every conditional in sensor.aiml has a bare default branch	An unreadable sensor can never produce silence or a false all-clear	24 of 24
No duplicate AIML patterns	A duplicate silently shadows the earlier category	0 duplicates in 362 patterns
"unknown" appears in no alerting state set	A dead sensor can neither raise an alarm nor clear one	Confirmed

A failed ToF read encodes as 0xFFFF, never 0	0 is indistinguishable from an object touching the lens — exactly the reading an intrusion produces	Confirmed
The ESP-NOW frame size is derived from its format string	The 10-versus-12-byte silent drop cannot recur	Confirmed
Duplicated firmware files are checked before every flash	Six files exist in two or three upload folders; check_pack.py fails loudly if they diverge	Confirmed

Table 30: Structural correctness invariants

Behavioral correctness of the safety answer. The most important single behaviour in the system is what GENIE says when it is asked "check all sensors" while part of its own

smoke	flame	fall	motion	GENIE answers
clear	clear	idle	false	"All clear."
clear	clear	idle	true	"One of my safety sensors is slightly elevated."
clear	clear	idle	unknown	"I do not have a full safety reading right now."
alarm	clear	idle	unknown	"There is a safety alarm active." — an alarm outranks a blind spot

Table 31: Safety answer truth table

The initially wrong one is the third row. When the ToF array was disconnected, each safety channel showed 'unknown', none matched an alarm test, and the reply ended up in the 'everything looks fine' fallback, no smoke, fire or falls detected" system had no visibility.

Path	Chain	Time
Fall, flame	Node 1 FSM confirms → serial line → bridge → API → alert decision	50–150 ms
Smoke alarm	As above, plus the deliberate 4 s MQ-2 debounce	≈ 4 s
Motion / presence	Node 2 event → Node 1 → 2 s tick → bridge → debounce	≈ 6 s
Conversation	Wake word → Whisper → Mistral-7B → Piper	8–15 s
On-board sensor confirmation	Three consecutive agreeing polls before a status changes at all	Flame ≈ 150 ms, smoke ≈ 600 ms

Display	ILI9341 parallel driver, soak test	2,477 kpx/s, 118 fps

Table 32: Measured latency by path

9. CONCLUSION

GENIE set out to demonstrate that a voice assistant and a home safety monitor can be one device, built upon commodity microcontrollers, all of whose inference stages can be executed locally and leave nothing of value out of the house. It does. Wake-word detection, speech recognition, intent routing, language generation and speech synthesis are all performed upon one consumer laptop with the internet disconnected while six ESP32-S3 boards share the I/O.

The strongest technical result came from the on-device machine learning. A 232-parameter classifier trained upon 1532 windows of accelerometer data runs in pure Micro Python on the face board, sharing a feature extractor with its training script, so that train/serve skew cannot happen. Accuracy rose from 31.4% at an initially-diagnosed level through the use of diagnosis rather than tuning, reaching 72.6% on an unseen session. A random split would have claimed 88.3%, but overlapping windows make that a measure of memorization rather than generalization. A 0.70 confidence gate boosts accuracy of answered windows to 82.6%, at the expense of being silent on 22.2% of them. This is a deliberate sacrifice, because silence is harmless and a wrong reaction is not.

The same principle informs the safety layer: a missing sensor reading is presented as an 'unknown' and never as a 'clear', and that only one component may raise an emergency. Verification was treated as a deliverable in its own right, with 104 assertions across 5 automated suites running without any hardware attached. Of 29 backlog items, 21 are complete and demonstrated, 5 are implemented but blocked by factors outside the software, and 3 were out of scope from the start. The blockers are an intermittent power-supply fault in the time-of-float sensor array, an unregistered SIM, and one display channel not yet proved on hardware.

GENIE is therefore a working proof that private voice interaction, integrated safety monitoring and on-device machine learning can share one low-cost platform, each backed up by a measurement. It is not a medical device: the fall detector was tuned on 1 person and one surface, the classifier was trained on a single unit, and the gas baseline has not been calibrated against a real combustion source. These limitations are made plain because a safety system that overstates its reliability is worse than one which states its own limits.

10. FUTURE WORK

Finishing what already exists.

- Power supply for the cellular module. Every question left about the emergency path comes down to whether the supply can handle a 2 A transmission burst. Testing the module with a source that has very low internal resistance is a brief test, and there is only one left that could change the choice to disable calling. If it works, the call path is restored by switching one configuration flag, because the firmware is complete.
- Confirmed message delivery. Delivery-report handling is implemented but has never been seen working, and the destination number is disputed between two configuration files. Aligning the number and seeing one report is the fastest way to verify the emergency path.
- Fall detection at its intended sample rate. The detector needs a sample every 20 MS but currently gets one about every 50 MS, so it is missing the impact spikes it is designed for. Moving the detector off of the display loop, or onto a timer interrupt, is necessary before it can be turned back on. It should also be clearly noted that the current detector is sensing the device falling, not a person.
- Gas and smoke calibration. The sensors need to be powered for 24 to 48 hours before their baseline is significant enough to be meaningful, and the baseline has to be recorded against a real source before any smoke reading can be trusted.
- The two remaining evaluation gates. The browser dashboard (EP5-01) and battery-backed operation (EP5-02, NFR-07) are listed in the backlog and have not been started.
- Display feedback for the follow-up conversation state. The host already sends the signals; the firmware has no handler for them so nothing is displayed. Unknown signals are safely ignored, which is why this has gone unnoticed.

Resolving what remains unproven.

- Decide on your beamformer. Neither channel sounds provenly better, and the status quo keeps the more complex option as the only valid choice until a longer matched false-accept count on both channels demonstrates decisiveness. If one mic proves itself equal, pick it for its simplicity, lack of off-axis comb filtering, and saving of a per-sample processing loop.
- Lower the false-accept rate. About one activation per minute in a room with conversation is the most egregious current deficiency. The first and cheapest diagnostic is an untried one: track the score of each false accept, since a false accept at 0.98 and one at 0.66 are problems of different levels requiring different solutions.

- Retrain the wake-word model with sub-phrases as explicit negatives. Partial phrases achieve scores up to 0.59, uncomfortably close to the 0.65 threshold; training against them directly would widen the margin without loss of far-field sensitivity.
- Address the remaining close-range miss rate. About one attempt in ten fails at half a meter, where signal quality cannot be the cause. The leading hypothesis is that resetting the detector's feature context between utterances leaves the next one scored against a cold buffer, and a spaced repetition test would confirm or falsify that.

Extensions.

Reviewing the original set of features to be implemented, and comparing them to the current state of the project.

- Vibration assisted activity classification. The “picked up” class has a recall of about 29% which is not great because the accelerometer cannot differentiate between lift and touch. The vibration switches.
- Already fitted to the board give exactly the signal missing, and adding them is the identified fix rather than a speculative improvement.
- A tap sequence command channel. Two, three and four taps on the enclosure are discernible events, giving a non-verbal input to those who cannot talk reliably, or need to use a noisy room.
- Barge in. To interrupt a reply in mid-sentence, require acoustic echo handling, and was deliberately omitted from this version but is the most obvious gap in conversational feel.
- Return of the audio path on the emergency call. Three approaches were tried and each closed for a documented reason; the most cost-effective way back is acoustic coupling, requiring the module's microphone connection fixed. Electrical injection would require a printed circuit board with a separately regulated supply and a ground plane the design is correct and documented, only the layout is missing.
- Speaker differentiation. GENIE cannot tell who is speaking, limiting both personalization and any user-specific access controls over appliance commands.
- Camera-based verification. Visual confirmation of a sensor alarm was specified early and dropped for technical reasons. If revisited, it needs a new justification rather than a continuation of the original one.

11. BIBLIOGRAPHY

11.1 Books

- [1] Karl E Wiegers and Joy Beatty. 2013. Software Requirements 3. Microsoft Press, USA.
- [2] Craig Larman. 2001. Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and the Unified Process (2nd. ed.). Prentice Hall PTR, USA.
- [3] Martin Fowler. 2003. UML Distilled: A Brief Guide to the Standard Object Modeling Language (3rd. ed.). Addison-Wesley Longman Publishing Co., Inc., USA.

11.2 Journals

- [4] Bulling, U. Blanke, and B. Schiele. 2014. A tutorial on human activity recognition using body-worn inertial sensors. ACM Computing Surveys 46, 3 (January 2014), 1–33.
- [5] D. Lara and M. A. Labrador. 2013. A survey on human activity recognition using wearable sensors. IEEE Communications Surveys & Tutorials 15, 3 (2013), 1192–1209.
- [6] J. Li, L. Deng, Y. Gong, and R. Haeb-Umbach. 2014. An overview of noise-robust automatic speech recognition. IEEE/ACM Transactions on Audio, Speech, and Language Processing 22, 4 (April 2014), 745–777.

11.3 Articles

- [7] R. C. Wallace. 2009. The anatomy of A.L.I.C.E. In Parsing the Turing Test, R. Epstein, G. Roberts, and G. Beber (Eds.). Springer, Dordrecht, 181–210.
- [8] Expressive Systems. 2023. ESP32-S3 Series Datasheet, Version 1.6. Expressive Systems (Shanghai) Co., Ltd.

11.4 Other References

- [9] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in Proc. 40th Int. Conf. Machine Learning (ICML), 2023. arXiv:2212.04356.
- [10] SYSTRAN, "faster-whisper: Faster Whisper transcription with CTranslate2." [Online]. Available: https://github.com/SYSTRAN/faster-whisper
- [11] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, et al., "Mistral 7B," arXiv:2310.06825, 2023.
- [12] Ollama, "Ollama: Run large language models locally." [Online]. Available: https://ollama.com/
- [13] Rhasspy, "Piper: A fast, local neural text-to-speech system." [Online]. Available: https://github.com/rhasspy/piper

- [14] D. Scripka, "openWakeWord: An open-source wake-word detection library." [Online]. Available: https://github.com/dscripka/openWakeWord
- [15] InvenSense, "INMP441: Omnidirectional microphone with bottom port and I²S digital output," datasheet.
- [16] InvenSense, "MPU-6000 and MPU-6050 product specification," Rev. 3.4, 2013.
- [17] Hanwei Electronics, "MQ-2 semiconductor sensor for combustible gas," datasheet.
- [18] STMicroelectronics, "VL53L1X: Time-of-flight ranging sensor," datasheet.
- [19] SIMCom Wireless Solutions, "A7670 series hardware design," V1.03.
- [20] Micro Python, "Micro Python documentation: ESP32 port." [Online]. Available: https://docs.micropython.org/
- [21] Maxim Integrated (Analog Devices), "MAX98357A/MAX98357B: PCM input class D audio power amplifier," datasheet.
- [22] S. Ramírez, "Fast API." [Online]. Available: https://fastapi.tiangolo.com/
- [23] J. Wiseman, "py-webrtcvad: Python interface to the WebRTC voice activity detector." [Online]. Available: https://github.com/wiseman/py-webrtcvad
- [24] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [25] G. Chen, C. Parada, and G. Hei gold, "Small-footprint keyword spotting using deep neural networks," in Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP), 2014, pp. 4087–4091.
- [26] T. N. Sainath and C. Parada, "Convolutional neural networks for small-footprint keyword spotting," in Proc. INTERSPEECH, 2015, pp. 1478–1482.
- [27] P. Warden, "Speech Commands: A dataset for limited-vocabulary speech recognition," arXiv:1804.03209, 2018.
- [28] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "MCUNet: Tiny deep learning on IoT devices," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020.
- [29] C. Knapp and G. Carter, "The generalized correlation method for estimation of time delay," IEEE Trans. Acoustics, Speech, and Signal Processing, vol. 24, no. 4, pp. 320–327, Aug. 1976.

12. APPENDIX

12.1 Glossary of terms

Term	Definition
Wake word	The fixed phrase (“Hey Genie”) that must be detected before the system begins recording. It exists so the microphone can be continuously monitored at low cost without continuously recording.
Wake threshold	The model score above which a block of audio is treated as containing the wake phrase. Set to 0.65 in this system.
Run-length gate	A requirement that the wake score stay above threshold for several consecutive analysis blocks. It rejects brief impulsive noises without raising the threshold and so without sacrificing detection range.
Voice activity detection (VAD)	The stage that decides, frame by frame, whether audio contains speech, used to decide when a spoken command has ended.
RMS floor	A minimum short-window audio level a frame must exceed before it can count as speech, used alongside VAD to reject quiet non-speech sounds.
Pre-roll	A short buffer of audio retained from just before a wake-word detection and prepended to the recording, so the first word of a command is never clipped.
Follow-up window	A brief period after the system finishes speaking during which sustained speech alone opens a new capture, so a conversation does not require the wake phrase on every turn.
Chain cap	A hard limit on how many consecutive wake-word-free captures may occur. It is a structural guarantee against an acoustic feedback loop, as opposed to a probabilistic defence such as a timing margin.
Microphone mute	Suppression of all audio processing while the system is speaking, so the system cannot hear and respond to its own voice.
Beamforming (delay-and-sum)	Combining two microphone signals to favors sound arriving from directly in front. Measured in this project and found to give no
Confidence selection	Choosing between the two microphone channels by comparing the transcription confidence of each, rather than by comparing loudness. Clarity and volume are not the same thing.
Deterministic-first routing	Checking a transcribed command against fixed patterns before consulting the language model, so that device commands are fast and predictable and the model can never invent a sensor reading.

Edge AI	Running a trained model on the microcontroller itself rather than on the host, so that a classification does not depend on the network or the host being available.
Held-out-session evaluation	Testing a model on complete recording sessions it has never seen, rather than on a random split. Necessary here because overlapping analysis windows make a random split a test of memorization.
Train/serve skew	The failure mode in which features are computed differently during training and during deployment, so a model is accurate only on data that never occurs in the field.
Edge-triggered alerting	Raising an alert on the transition into a hazard state rather than repeatedly for as long as the state persists.
Debounce	A required period of stable reading before a sensor change is accepted, used to stop a twitchy sensor raising repeated alarms.
Unknown state	An explicit sensor state distinct from “clear”. A faulty or disconnected sensor reports unknown, which can never satisfy an alarm condition and can never be mistaken for an all-clear.
Zero-configuration discovery	The mechanism by which a node finds the host by broadcasting on the local network, so no address has to be configured by hand.
Active-LOW relay	A relay module that switches on when its control pin is driven low rather than high; relevant because the firmware's logic is inverted with respect to the intuitive reading.
Robo Eyes	The animated eye expressions rendered on the node display, used as a non-verbal indication of system state for users who benefit from visual rather than textual feedback.

12.2 Pre-Requisites

The following must be in place before GENIE can be deployed and run. These are drawn from the development and deployment setup of Chapter 6 and the external system dependencies of Chapter 2.

Host machine.

Requirement	**Detail**
Operating system	Windows 11
Runtime	Python 3.11
Speech recognition	faster-whisper, with the selected model weights downloaded in advance
Wake word	openWakeWord with ONNX Runtime, and the trained hey_genie. onnx model present

Speech synthesis	Piper, with the en_US voice model downloaded
Language model	Ollama installed and running, with the Mistral-7B model pulled before GENIE starts
Intent engine	PyAIML, with relay. aiml and sensor. aiml present
API layer	Fast API with Uvicorn
Supporting libraries	NumPy, scikit-learn, webrtcvad, pySerial, pytest
Audio hardware	A working audio subsystem; no cloud audio service is used or required

Network:

- A dedicated wireless access point should be used for the nodes. The host and every Wi-Fi connected node need to be on the same subnet.
- No part of the voice or safety pipeline requires an internet connection. An internet connection is needed only for the cellular emergency path, which relies on the mobile network rather than the local one.
- Inbound firewall rules have to allow the host's Python process to accept connections on the audio, control, telemetry, and discovery ports. During development, blocked inbound rules were repeatedly misidentified as hardware faults, so they are the first thing to check if a node fails to connect.
- It is strongly advised to use a fixed host address. Nodes using discovery can handle an address change; any node with a hardcoded address cannot.

Hardware nodes.

Node	**Core components**	**Firmware**
Microphone node	ESP32-S3, two INMP441 IZS MEMS microphones	Arduino-ESP32 (C++)
Speaker node	ESP32-S3, MAX98357A Class-D amplifier, 5 W 4 fi speaker	Arduino-ESP32 (C++)
Relay / IoT node	ESP32-S3, active-LOW relay module, WS2812 status LED	Arduino-ESP32 (C++)
Sensor node	ESP32-S3 with octal PSRAM, two MQ-2 gas sensors, flame sensor, DHT11, two MPU-6050 IMUs, two SW420 vibration switches, ILI9341 display	MicroPython
Modem node	ESP32-S3, A7670E LTE module, registered SIM	MicroPython

Hardware conditions that must be met before use.

- The sensor node must have the correct Micro Python build flashed onto it. A build lacking octal PSRAM support yields only about 100 KB of free memory instead of 8 MB, and shows up as unexplained out-of-memory errors.
- The gas sensors need to be powered continuously for 24 to 48 hours before their baseline is recorded. A baseline captured from a cold sensor causes the system to go into permanent false alarms.
- All boards and any bench power supply must share a common ground; otherwise, analog readings drift between states without a valid reference.
- The cellular module needs a power supply that can handle its transmit burst current. An inadequate supply appears as protocol errors rather than a power fault, which is why it is hard to diagnose.
- The SIM in the modem node must be registered with the national regulator, or the network will block the device once the grace period ends.

Development tools.

- Thonny, Esp tool, and MP remote for Micro Python nodes. Only one tool can occupy a serial port at a time.
- The Arduino-ESP32 core, or Platform IO, for the C++ nodes.
- Git for version control of both the host codebase and the firmware.
- pytest, to run the automated test suites before spending hardware time.